ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC & THUẬT MÁY TÍNH

XỬ LÝ TÍN HIỆU SỐ (CO2036)

LAB 3

Discrete-Time Signal Operations and System Analysis Using Scilab

Lớp: L03 - Học Kỳ HK252
Ngày nộp: 27/03/2026

GVHD: Thầy Phạm Công Thái

Sinh Viên	MSSV
Phạm Công Võ	2313946
Nguyễn Anh Thư	2313385

TP.Hồ Chí Minh, Tháng 03/2026

PHÂN CÔNG NHIỆM VỤ VÀ KẾT QUẢ THỰC HIỆN ĐỀ TÀI CỦA TỪNG THÀNH VIÊN NHÓM

STT	Họ và tên	MSSV	Nhiệm vụ	Kết quả	Chữ ký
1	Nguyễn Anh Thư	2313385	• Hiện thực bài tập 2, 4, 6 và 2.4, 2.6 • Viết báo cáo	100%	
2	Phạm Công Võ	2313946	• Hiện thực bài tập 1, 3, 5 và 2.1, 2.2 • Viết báo cáo	100%	

Mục lục

1 Giới thiệu **4**

2 Mục tiêu thực hiện **4**

3 Phương pháp thực hành **4**

4 Thực hiện các phép biến đổi tín hiệu rời rạc bằng Scilab **5**

4.1 Exercise 1: Thực hiện phép trễ tín hiệu (Delay) 5
4.2 Exercise 2: Thực hiện phép dịch tiến tín hiệu (Advance) 8
4.3 Exercise 3: Thực hiện phép đảo tín hiệu (Folding) 11
4.4 Exercise 4: Thực hiện phép cộng tín hiệu (Signal Addition) 14
4.5 Exercise 5: Thực hiện phép nhân tín hiệu (Signal Multiplication) 17
4.6 Exercise 6: Thực hiện phép tích chập (Convolution) 20

5 Giải các bài tập tín hiệu rời rạc trong tài liệu tham khảo **23**

5.1 Problem 2.1: Phân tích tín hiệu rời rạc 23
5.2 Problem 2.2: Biến đổi tín hiệu rời rạc 34
5.3 Problem 2.4: Phân rã tín hiệu thành phần chẵn và lẻ 54
5.4 Problem 2.6: Time Invariance of Discrete-Time Systems 57

Tài liệu tham khảo **66**

Danh sách hình vẽ

1 Đồ thị tín hiệu gốc $x(n)$ và tín hiệu trễ $y(n) = x(n-1)$ 7
2 Đồ thị tín hiệu gốc $x(n)$ và tín hiệu nhanh $y(n) = x(n+1)$ 10
3 Đồ thị tín hiệu gốc $x(n)$ và tín hiệu gập $y(n) = x(-n)$ 13
4 Đồ thị tín hiệu $x_1(n)$, $x_2(n)$ và tín hiệu tổng $y(n) = x_1(n) + x_2(n)$ 16
5 Đồ thị tín hiệu $x_1(n)$, $x_2(n)$ và tín hiệu tích $y(n) = x_1(n) \cdot x_2(n)$ 19
6 Đồ thị tín hiệu vào $x(n)$, đáp ứng xung $h(n)$ và tín hiệu ra $y(n) = x(n) * h(n)$ 22
7 Đồ thị tín hiệu rời rạc $x(n)$ 25
8 Kết quả Fold $x(n)$ rồi Delay 4 mẫu: $y(n) = x(-n+4)$ 27
9 Kết quả Delay 4 mẫu rồi Fold $x(n)$: $y(n) = x(-n-4)$ 29
10 Đồ thị tín hiệu $x(-n+4)$ 31
11 Tín hiệu gốc $x(n)$ và tín hiệu trễ $x(n-2)$ 36
12 Tín hiệu gốc $x(n)$ và tín hiệu $x(4-n)$ thu được sau khi Fold rồi Delay 4 mẫu. 38
13 Tín hiệu gốc $x(n)$ và tín hiệu $x(n+2)$ 40
14 Tín hiệu $x(n)$, hàm bước $u(2-n)$ và tín hiệu tích $y(n) = x(n) \cdot u(2-n)$ 43
15 Tín hiệu $x(n-1)$ và tích $y(n) = x(n-1) \cdot \delta(n-3)$ 45
16 Tín hiệu gốc $x(n)$ và tín hiệu $x(n^2)$ sau khi tái lấy mẫu phi tuyến 47
17 Tín hiệu gốc $x(n)$, tín hiệu $x(-n)$ và thành phần chẵn $x_e(n)$ 50
18 Tín hiệu gốc $x(n)$, tín hiệu $x(-n)$ và thành phần lẻ $x_o(n)$ 53
19 Các tín hiệu $x(n)$, $y(n)$, $y(n-2)$, $x_2(n)$ và $y_2(n)$ 61
20 Các tín hiệu $x(n)$, $y(n)$, $y(n-2)$, $x_2(n)$ và $y_2(n)$ với $\mathcal{T}[x(n)] = x(n) - x(n-1)$ và $k = 2$. 63
21 Các tín hiệu $x(n)$, $y(n)$, $y(n-2)$, $x_2(n)$ và $y_2(n)$ với $\mathcal{T}[x(n)] = n \cdot x(n)$ và $k = 2$ 65

1 Giới thiệu

Xử lý tín hiệu số (Digital Signal Processing - DSP) là một lĩnh vực quan trọng trong kỹ thuật điện – điện tử và công nghệ thông tin, cho phép phân tích và xử lý các tín hiệu rời rạc theo thời gian. Các tín hiệu này xuất hiện trong nhiều ứng dụng thực tế như truyền thông số, xử lý âm thanh, xử lý hình ảnh và các hệ thống nhúng.

Để hiểu rõ cách các hệ thống xử lý tín hiệu hoạt động, việc nắm vững các phép biến đổi cơ bản của tín hiệu rời rạc là rất cần thiết, bao gồm dịch tín hiệu theo thời gian, đảo tín hiệu và các phép toán giữa nhiều tín hiệu.

Trong bài thực hành Lab 3, sinh viên sử dụng phần mềm Scilab để xây dựng và kiểm chứng các phép biến đổi tín hiệu như delay, advance, fold, cộng tín hiệu, nhân tín hiệu và tích chập (convolution). Ngoài ra, một số bài tập trong tài liệu tham khảo cũng được thực hiện nhằm củng cố kiến thức lý thuyết về tín hiệu rời rạc và các phép toán trong xử lý tín hiệu số.

2 Mục tiêu thực hiện

Bài thực hành này được thực hiện nhằm đạt được các mục tiêu sau:

- Hiểu rõ khái niệm và cách biểu diễn tín hiệu rời rạc theo thời gian.
- Nắm vững các phép biến đổi cơ bản của tín hiệu rời rạc như dịch tín hiệu (delay, advance) và đảo tín hiệu (fold).
- Thực hiện các phép toán giữa các tín hiệu rời rạc bao gồm phép cộng, phép nhân và phép tích chập.
- Xây dựng và triển khai các hàm xử lý tín hiệu bằng ngôn ngữ lập trình Scilab.
- Sử dụng Scilab để mô phỏng và trực quan hóa tín hiệu thông qua đồ thị.
- Củng cố kiến thức lý thuyết về tín hiệu rời rạc thông qua việc giải các bài tập liên quan trong tài liệu tham khảo.

3 Phương pháp thực hành

- Nghiên cứu các khái niệm lý thuyết liên quan đến tín hiệu rời rạc và các phép biến đổi cơ bản trong xử lý tín hiệu số thông qua tài liệu học tập và tài liệu tham khảo.
- Xây dựng các hàm xử lý tín hiệu trong môi trường Scilab, bao gồm các phép biến đổi như delay, advance, fold, cũng như các phép toán giữa hai tín hiệu như cộng, nhân và tích chập.
- Tạo các tín hiệu mẫu dưới dạng vector và sử dụng các hàm đã xây dựng để thực hiện các phép biến đổi và phép toán tương ứng.
- Sử dụng Scilab để vẽ đồ thị biểu diễn tín hiệu trước và sau khi biến đổi nhằm trực quan hóa kết quả xử lý.
- Phân tích và so sánh các kết quả thu được để kiểm tra tính chính xác của các hàm đã xây dựng.
- Tổng hợp kết quả thực hiện và trình bày trong báo cáo thực hành.

4 Thực hiện các phép biến đổi tín hiệu rời rạc bằng Scilab

4.1 Exercise 1: Thực hiện phép trễ tín hiệu (Delay)

Yêu cầu đề bài

- **Xây dựng hàm Scilab:**

  `function [yn, yorigin] = delay(xn, xorigin, k)`

  để thực hiện phép trễ tín hiệu rời rạc theo công thức:

$$y(n) = x(n-k), \quad k > 0$$

- **Ý nghĩa các tham số:**
  - `xn`: **vector biểu diễn tín hiệu rời rạc** $x(n)$
  - `xorigin`: **vị trí gốc của tín hiệu** $x(n)$
  - `k`: **số mẫu dịch**
  - `yn`: **vector biểu diễn tín hiệu đầu ra** $y(n)$
  - `yorigin`: **vị trí gốc của tín hiệu** $y(n)$
- **Sau khi thực hiện phép biến đổi, tín hiệu đầu vào** $x(n)$ **và tín hiệu kết quả** $y(n)$ **phải được hiển thị bằng đồ thị trong cùng một hình để so sánh.**
- **Ví dụ minh họa:**

  Xác định tín hiệu trễ

$$y(n) = x(n-1)$$

  với tín hiệu ban đầu:

$$x(n) = \{1, -2, 3\uparrow, 6\}$$

  Trong đó ký hiệu $\uparrow$ **biểu thị vị trí gốc của tín hiệu.**
- **Gọi hàm đã xây dựng:**

```
[yn, yorigin] = delay([1 -2 3 6], 3, 1)
```

- **Kết quả mong đợi:**
  - `yn = [1 -2 3 6]`
  - `yorigin = 2`
- **Đồng thời cần vẽ đồ thị biểu diễn tín hiệu** $x(n)$ **và** $y(n)$ **trong cùng một figure để minh họa kết quả của phép trễ tín hiệu.**

Mục tiêu

Xây dựng hàm Scilab thực hiện **phép toán trễ** trên tín hiệu rời rạc, được định nghĩa theo công thức:

$$y(n) = x(n-k), \quad k > 0$$

Phép trễ tín hiệu là phép dịch chuyển tín hiệu sang phải k bước trên trục thời gian rời rạc n.

Đặc điểm:

- Các giá trị mẫu **hoàn toàn không đổi**, chỉ có trục thời gian thay đổi.
- Gốc tọa độ dịch sang trái k đơn vị:

$$y_{\text{origin}} = x_{\text{origin}} - k$$

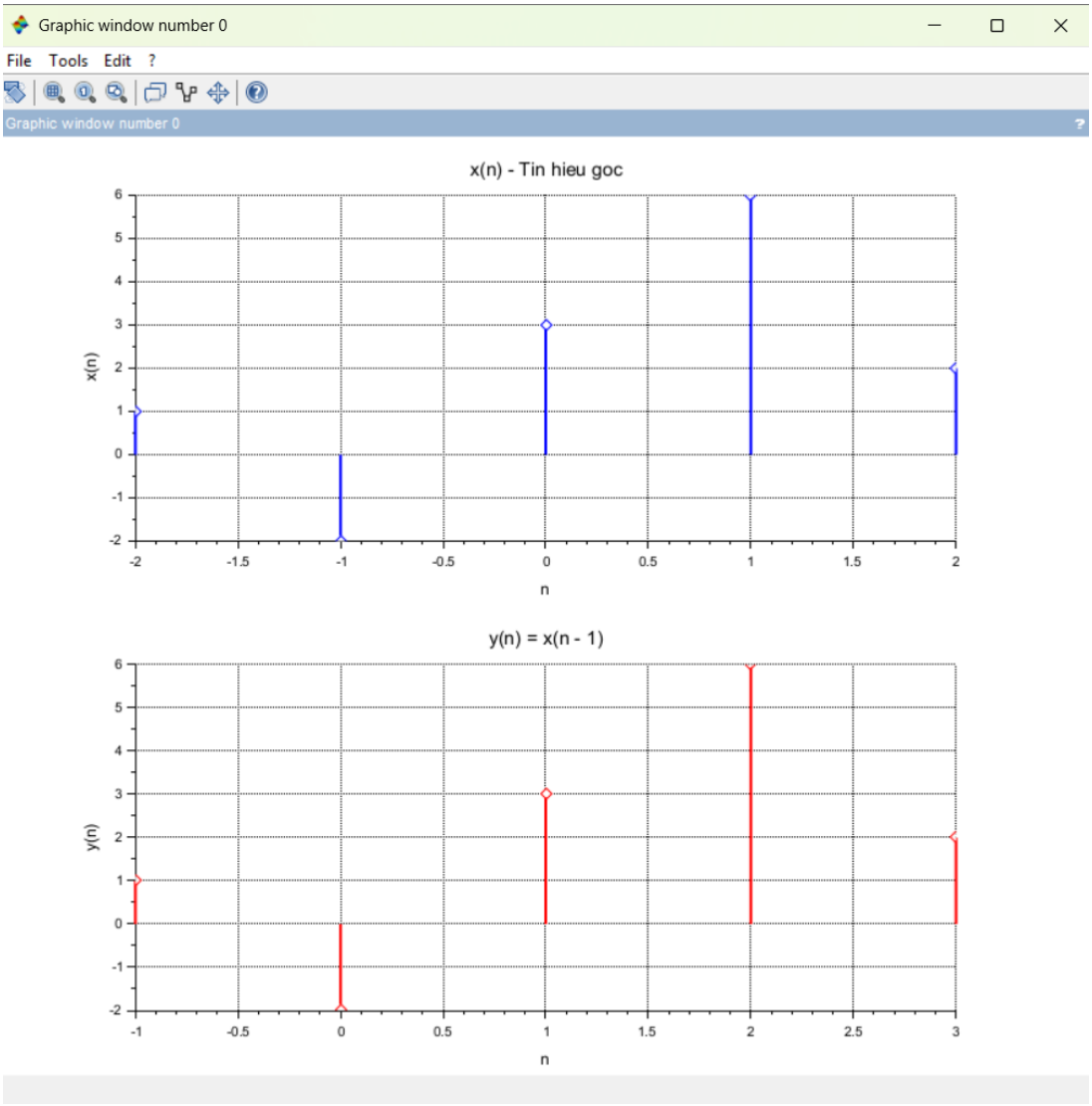
Cài đặt trên Scilab

```
// ============================================================
// FUNCTION : delay
// PURPOSE  : Perform delay operation y(n) = x(n - k)
// INPUTS   : xn      - sample values of input signal x(n)
//            xorigin - index of n=0 in vector xn (1-based)
//            k       - number of delay steps (k > 0)
// OUTPUTS  : yn      - sample values of output signal y(n)
//            yorigin - index of n=0 in vector yn (1-based)
// ============================================================
function [yn, yorigin] = delay(xn, xorigin, k)
    yn      = xn;                    // sample values unchanged
    yorigin = xorigin - k;           // origin index shifts left by k

    Nx = length(xn);
    nx = (1:Nx) - xorigin;           // time axis of x(n)
    ny = (1:Nx) - yorigin;           // time axis of y(n)

    scf(); clf();
    // Plot x(n) - original signal
    subplot(2,1,1);
    plot2d3(nx, xn, style=2); gce().children.thickness = 2;
    plot2d(nx, xn, style=-5); gce().children.mark_foreground = 2;
    xgrid(); xlabel('n'); ylabel('x(n)'); title('x(n) - Tin hieu goc');
    // Plot y(n) - delayed signal
    subplot(2,1,2);
    plot2d3(ny, yn, style=5); gce().children.thickness = 2;
    plot2d(ny, yn, style=-5); gce().children.mark_foreground = 5;
    xgrid(); xlabel('n'); ylabel('y(n)');
    title('y(n) = x(n - ' + string(k) + ')');
endfunction
```

```
// ------------------------------------------------------------
// Example: x(n) = {1, -2, 3^, 6, 2}, xorigin = 3, k = 1
// Result: yn = [1, -2, 3, 6, 2],  yorigin = 2
// ------------------------------------------------------------
[yn, yorigin] = delay([1,-2,3,6,2], 3, 1);
disp('yn␣='); disp(yn);
disp('yorigin␣='); disp(yorigin);
```

Kết quả Console và Đồ thị minh họa

```
--> exec('C:\Users\ASUS\Documents\Xử Lí Tín Hiệu Số\Lab\Lab3\Ex31.sci', -1)
 "yn ="
  1.  -2.   3.   6.   2.
 "yorigin ="
  2.
```



Hình 1: *Đồ thị tín hiệu gốc $x(n)$ và tín hiệu trễ $y(n) = x(n-1)$*

4.2 Exercise 2: Thực hiện phép dịch tiến tín hiệu (Advance)

Yêu cầu đề bài

- **Xây dựng hàm Scilab:**

  `function [yn, yorigin] = advance(xn, xorigin, k)`

  để thực hiện phép dịch tiến tín hiệu rời rạc theo công thức:

$$y(n) = x(n+k), \quad k > 0$$

- **Ý nghĩa các tham số:**
  - `xn`: **vector biểu diễn tín hiệu rời rạc** $x(n)$
  - `xorigin`: **vị trí gốc của tín hiệu** $x(n)$
  - `k`: **số mẫu dịch**
  - `yn`: **vector biểu diễn tín hiệu đầu ra** $y(n)$
  - `yorigin`: **vị trí gốc của tín hiệu** $y(n)$

- **Sau khi thực hiện phép biến đổi, tín hiệu đầu vào** $x(n)$ **và tín hiệu kết quả** $y(n)$ **phải được hiển thị bằng đồ thị trong cùng một hình để thuận tiện cho việc so sánh.**

- **Ví dụ minh họa:**

  Xác định tín hiệu tiến

$$y(n) = x(n+1)$$

  với tín hiệu ban đầu:

$$x(n) = \{1, -2, 3\uparrow, 6\}$$

  trong đó ký hiệu $\uparrow$ **biểu thị vị trí gốc của tín hiệu.**

- **Gọi hàm đã xây dựng:**

```
[yn, yorigin] = advance([1 -2 3 6], 3, 1)
```

- **Kết quả mong đợi:**
  - `yn = [1 -2 3 6]`
  - `yorigin = 4`

- **Đồng thời cần vẽ đồ thị biểu diễn tín hiệu** $x(n)$ **và** $y(n)$ **trong cùng một figure để minh họa sự thay đổi của tín hiệu sau khi thực hiện phép dịch tiến.**

Mục tiêu

Xây dựng hàm Scilab thực hiện phép toán trên tín hiệu rời rạc, được định nghĩa theo công thức:

$$y(n) = x(n+k), \quad k > 0$$

Phép dịch chuyển tín hiệu sang trái k bước trên trục thời gian rời rạc n, ngược chiều với phép trễ.
Đặc điểm:

- Các giá trị mẫu **hoàn toàn không đổi**, chỉ có trục thời gian thay đổi.
- Gốc tọa độ dịch sang phải k đơn vị (index tăng):

$$y_{\text{origin}} = x_{\text{origin}} + k$$

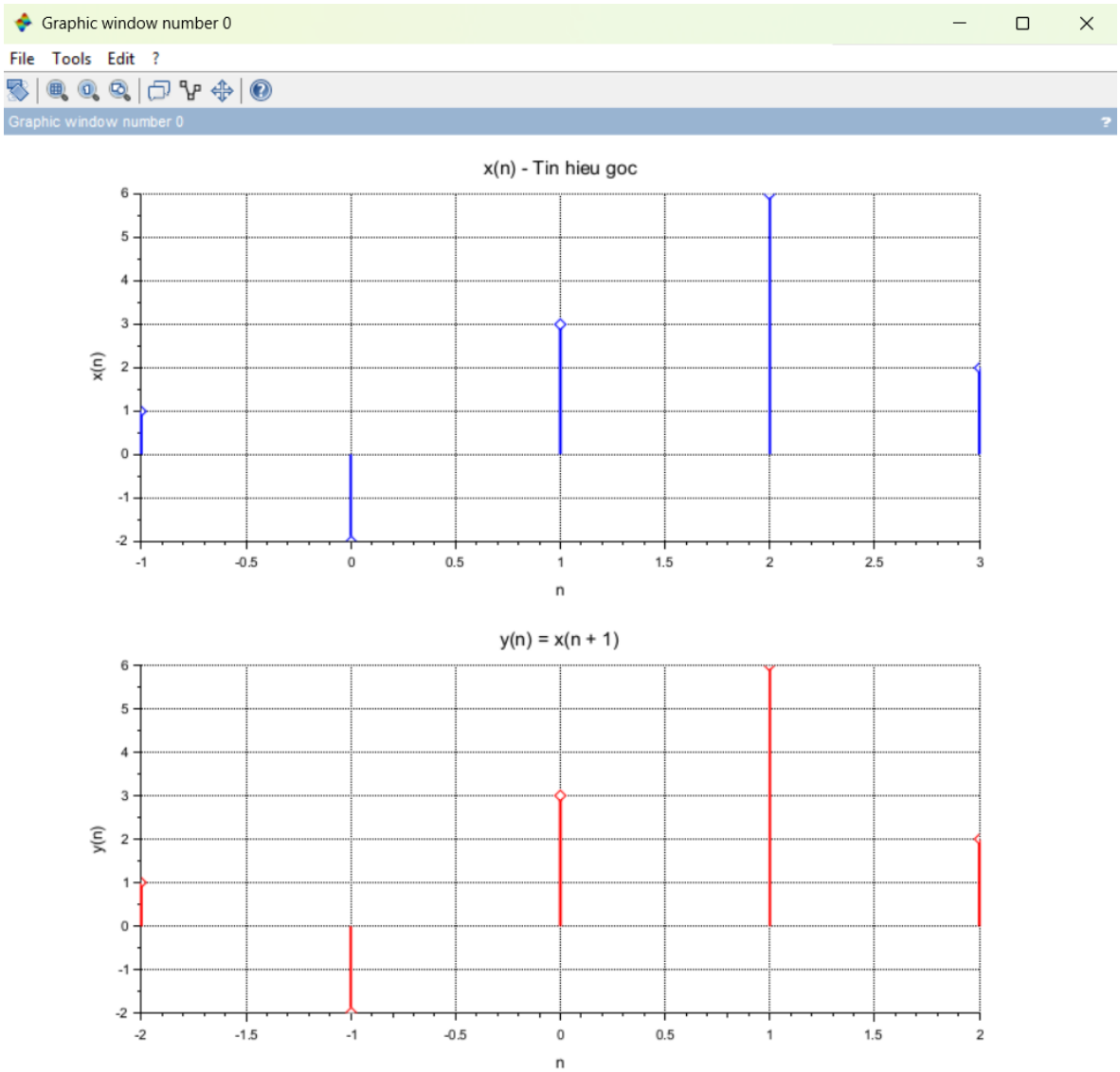
Cài đặt trên Scilab

```
// ============================================================
// FUNCTION : advance
// PURPOSE  : Perform advance operation y(n) = x(n + k)
// INPUTS   : xn       - sample values of input signal x(n)
//            xorigin  - index of n=0 in vector xn (1-based)
//            k        - number of advance steps (k > 0)
// OUTPUTS  : yn       - sample values of output signal y(n)
//            yorigin  - index of n=0 in vector yn (1-based)
// ============================================================
function [yn, yorigin] = advance(xn, xorigin, k)
    yn      = xn;                    // sample values unchanged
    yorigin = xorigin + k;           // origin index shifts right by k

    Nx = length(xn);
    nx = (1:Nx) - xorigin;           // time axis of x(n)
    ny = (1:Nx) - yorigin;           // time axis of y(n)

    scf(); clf();
    // Plot x(n) - original signal
    subplot(2,1,1);
    plot2d3(nx, xn, style=2); gce().children.thickness = 2;
    plot2d(nx, xn, style=-5); gce().children.mark_foreground = 2;
    xgrid(); xlabel('n'); ylabel('x(n)'); title('x(n) - Tin hieu goc');
    // Plot y(n) - advanced signal
    subplot(2,1,2);
    plot2d3(ny, yn, style=5); gce().children.thickness = 2;
    plot2d(ny, yn, style=-5); gce().children.mark_foreground = 5;
    xgrid(); xlabel('n'); ylabel('y(n)');
    title('y(n) = x(n + ' + string(k) + ')');
endfunction

```

```
// ------------------------------------------------------------
// Example: x(n) = {1, -2^, 3, 6, 2}, xorigin = 2, k = 1
// Result: yn = [1, -2, 3, 6, 2],  yorigin = 3
// ------------------------------------------------------------
[yn, yorigin] = advance([1,-2,3,6,2], 2, 1);
disp('yn␣='); disp(yn);
disp('yorigin␣='); disp(yorigin);
```

Kết quả Console và Đồ thị minh họa

```
--> exec('C:\Users\ASUS\Documents\Xử Lí Tín Hiệu Số\Lab\Lab3\Ex32.sci', -1)
 "yn ="
  1.  -2.   3.   6.   2.
 "yorigin ="
  3.
```



Hình 2: *Đồ thị tín hiệu gốc $x(n)$ và tín hiệu nhanh $y(n) = x(n+1)$*

4.3 Exercise 3: Thực hiện phép đảo tín hiệu (Folding)

Yêu cầu đề bài

- **Xây dựng hàm Scilab:**

  `function [yn, yorigin] = fold(xn, xorigin)`

  để thực hiện phép đảo tín hiệu rời rạc theo công thức:

$$y(n) = x(-n)$$

- **Ý nghĩa các tham số:**
  - `xn`: **vector biểu diễn tín hiệu rời rạc** $x(n)$
  - `xorigin`: **vị trí gốc của tín hiệu** $x(n)$ **trong vector**
  - `yn`: **vector biểu diễn tín hiệu đầu ra** $y(n)$
  - `yorigin`: **vị trí gốc của tín hiệu** $y(n)$ **trong vector**
- **Phép đảo tín hiệu (folding) thực hiện việc phản xạ tín hiệu qua trục** $n = 0$**. Nói cách khác, tín hiệu** $x(n)$ **được biến đổi thành** $x(-n)$**.**
- **Sau khi thực hiện phép biến đổi, tín hiệu đầu vào** $x(n)$ **và tín hiệu kết quả** $y(n)$ **phải được hiển thị bằng đồ thị trong cùng một hình để thuận tiện cho việc so sánh.**
- **Ví dụ minh họa:**

  Cho tín hiệu

$$x(n) = \{1,\ -2,\ 3\uparrow,\ 6\}$$

  trong đó ký hiệu $\uparrow$ **biểu thị vị trí gốc của tín hiệu.**
- **Gọi hàm đã xây dựng:**

```
[yn, yorigin] = fold([1 -2 3 6], 3)
```

- **Đồng thời cần vẽ đồ thị biểu diễn tín hiệu** $x(n)$ **và** $y(n)$ **trong cùng một figure để minh họa kết quả của phép đảo tín hiệu.**

Mục tiêu

Xây dựng hàm Scilab thực hiện phép toán trên tín hiệu rời rạc, được định nghĩa theo công thức:

$$y(n) = x(-n)$$

Phép gập tín hiệu là phép lật ngược tín hiệu quanh trục $n = 0$ trên trục thời gian rời rạc.

Đặc điểm:

- Vector các giá trị mẫu được đảo ngược thứ tự.
- Gốc tọa độ được tính lại theo công thức:

$$y_{\text{origin}} = N - x_{\text{origin}} + 1$$

  trong đó N là độ dài vector tín hiệu.

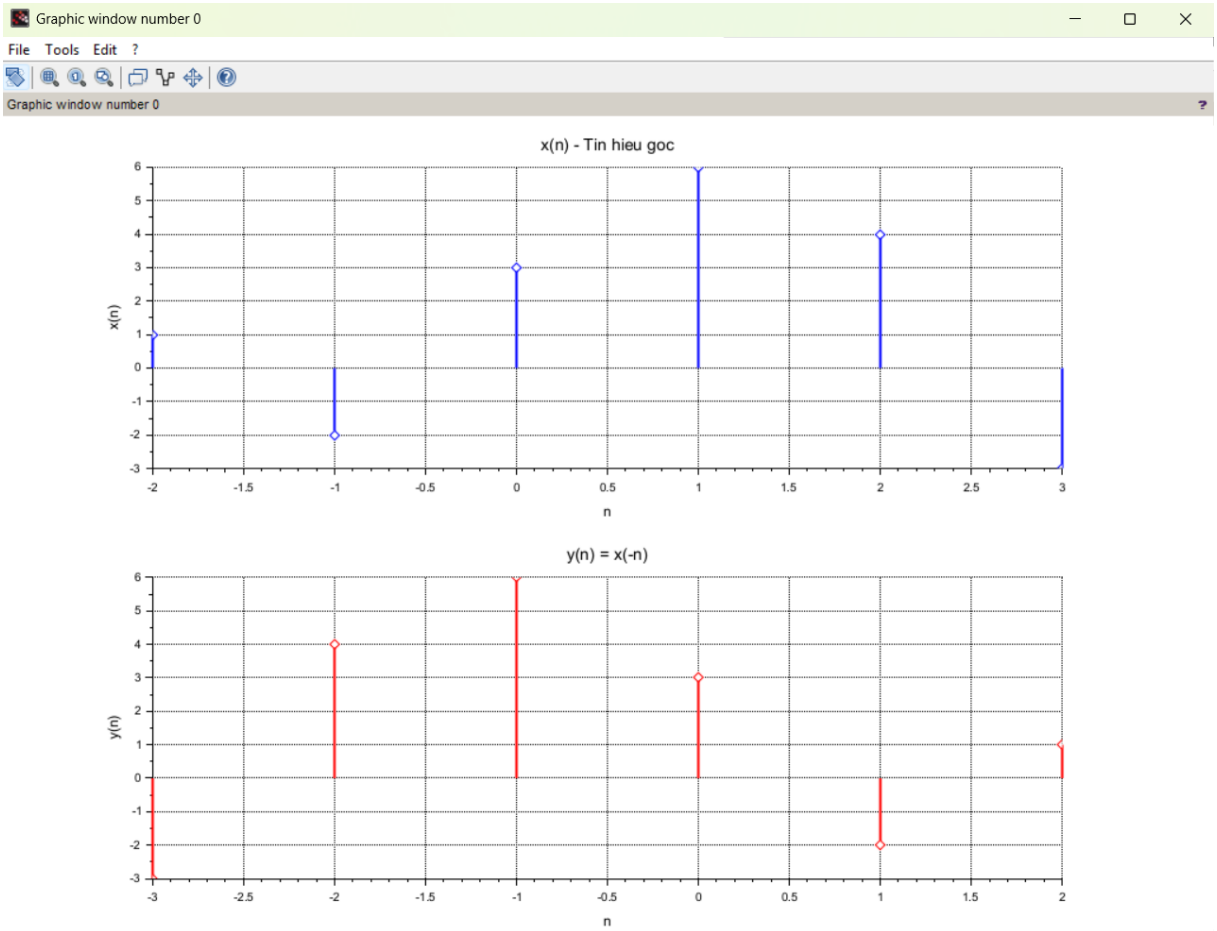
Cài đặt trên Scilab

```
// ============================================================
// FUNCTION : fold
// PURPOSE  : Perform folding operation y(n) = x(-n)
// INPUTS   : xn       - sample values of input signal x(n)
//            xorigin  - index of n=0 in vector xn (1-based)
// OUTPUTS  : yn       - reversed sample values of y(n)
//            yorigin  - index of n=0 in vector yn (1-based)
// ============================================================
function [yn, yorigin] = fold(xn, xorigin)
    Nx      = length(xn);
    yn      = xn($:-1:1);           // reverse the sample vector
    yorigin = Nx - xorigin + 1;     // mirror the origin position

    nx = (1:Nx) - xorigin;          // time axis of x(n)
    ny = (1:Nx) - yorigin;          // time axis of y(n)

    scf(); clf();
    // Plot x(n) - original signal
    subplot(2,1,1);
    plot2d3(nx, xn, style=2); gce().children.thickness = 2;
    plot2d(nx, xn, style=-5); gce().children.mark_foreground = 2;
    xgrid(); xlabel('n'); ylabel('x(n)'); title('x(n) - Tin hieu goc');
    // Plot y(n) - folded signal
    subplot(2,1,2);
    plot2d3(ny, yn, style=5); gce().children.thickness = 2;
    plot2d(ny, yn, style=-5); gce().children.mark_foreground = 5;
    xgrid(); xlabel('n'); ylabel('y(n)'); title('y(n) = x(-n)');
endfunction
```

```
// ------------------------------------------------------------
// Example: x(n) = {1, -2, 3^, 6, 4,-3}, xorigin = 3
// Result: yn = [-3, 4, 6, 3, -2, 1],  yorigin = 4
// ------------------------------------------------------------
[yn, yorigin] = fold([1,-2,3,6,4,-3], 3);
disp('yn␣='); disp(yn);
disp('yorigin␣='); disp(yorigin);
```

Kết quả Console và Đồ thị minh họa

```
--> exec('C:\Users\ASUS\Documents\Xử Lí Tín Hiệu Số\Lab\Lab3\Ex33.sci', -1)
 "yn ="
 -3.   4.   6.   3.  -2.   1.
 "yorigin ="
  4.
```



Hình 3: *Đồ thị tín hiệu gốc $x(n)$ và tín hiệu gập $y(n) = x(-n)$*

4.4 Exercise 4: Thực hiện phép cộng tín hiệu (Signal Addition)

Yêu cầu đề bài

- **Xây dựng hàm Scilab:**

  `function [yn, yorigin] = add(x1n, x1origin, x2n, x2origin)`

  để thực hiện phép cộng hai tín hiệu rời rạc theo công thức:

$$y(n) = x_1(n) + x_2(n)$$

- **Ý nghĩa các tham số:**
  - `x1n`: **vector biểu diễn tín hiệu** $x_1(n)$
  - `x2n`: **vector biểu diễn tín hiệu** $x_2(n)$
  - `x1origin`: **vị trí gốc của tín hiệu** $x_1(n)$
  - `x2origin`: **vị trí gốc của tín hiệu** $x_2(n)$
  - `yn`: **vector biểu diễn tín hiệu đầu ra** $y(n)$
  - `yorigin`: **vị trí gốc của tín hiệu** $y(n)$

- **Sau khi thực hiện phép toán, các tín hiệu** $x_1(n)$, $x_2(n)$ **và** $y(n)$ **phải được hiển thị bằng đồ thị trong cùng một figure để thuận tiện cho việc so sánh.**

- **Ví dụ minh họa:**

  Xác định tín hiệu

$$y(n) = x_1(n) + x_2(n)$$

  với

$$x_1(n) = \{0\uparrow, 1, 3, -2\}$$

$$x_2(n) = \{1, 1\uparrow, 2, 3\}$$

  trong đó ký hiệu $\uparrow$ **biểu thị vị trí gốc của tín hiệu.**

- **Gọi hàm đã xây dựng:**

```
[yn, yorigin] = add([0 1 3 -2], 1, [1 1 2 3], 2)
```

- **Kết quả mong đợi:**
  - `yn = [1 1 3 6 -2]`
  - `yorigin = 1`

- **Đồng thời cần vẽ đồ thị biểu diễn các tín hiệu** $x_1(n)$, $x_2(n)$ **và** $y(n)$ **trong cùng một figure để minh họa kết quả của phép cộng tín hiệu.**

Mục tiêu

Xây dựng hàm Scilab thực hiện phép toán trên tín hiệu rời rạc, được định nghĩa theo công thức:

$$y(n) = x_1(n) + x_2(n)$$

Phép cộng hai tín hiệu rời rạc thực hiện cộng từng giá trị mẫu tương ứng tại cùng thời điểm n.
Đặc điểm:

- Hai tín hiệu có thể có miền xác định khác nhau, cần mở rộng về miền hợp (union) trước khi cộng.
- Các vị trí không thuộc miền xác định của tín hiệu được **zero-padding** (bổ sung giá trị 0).
- Miền xác định của tín hiệu đầu ra:

$$n_{\min} = \min(n_{\min}^{(1)},\ n_{\min}^{(2)}), \quad n_{\max} = \max(n_{\max}^{(1)},\ n_{\max}^{(2)})$$

- Gốc tọa độ đầu ra:

$$y_{\text{origin}} = 1 - n_{\min}$$

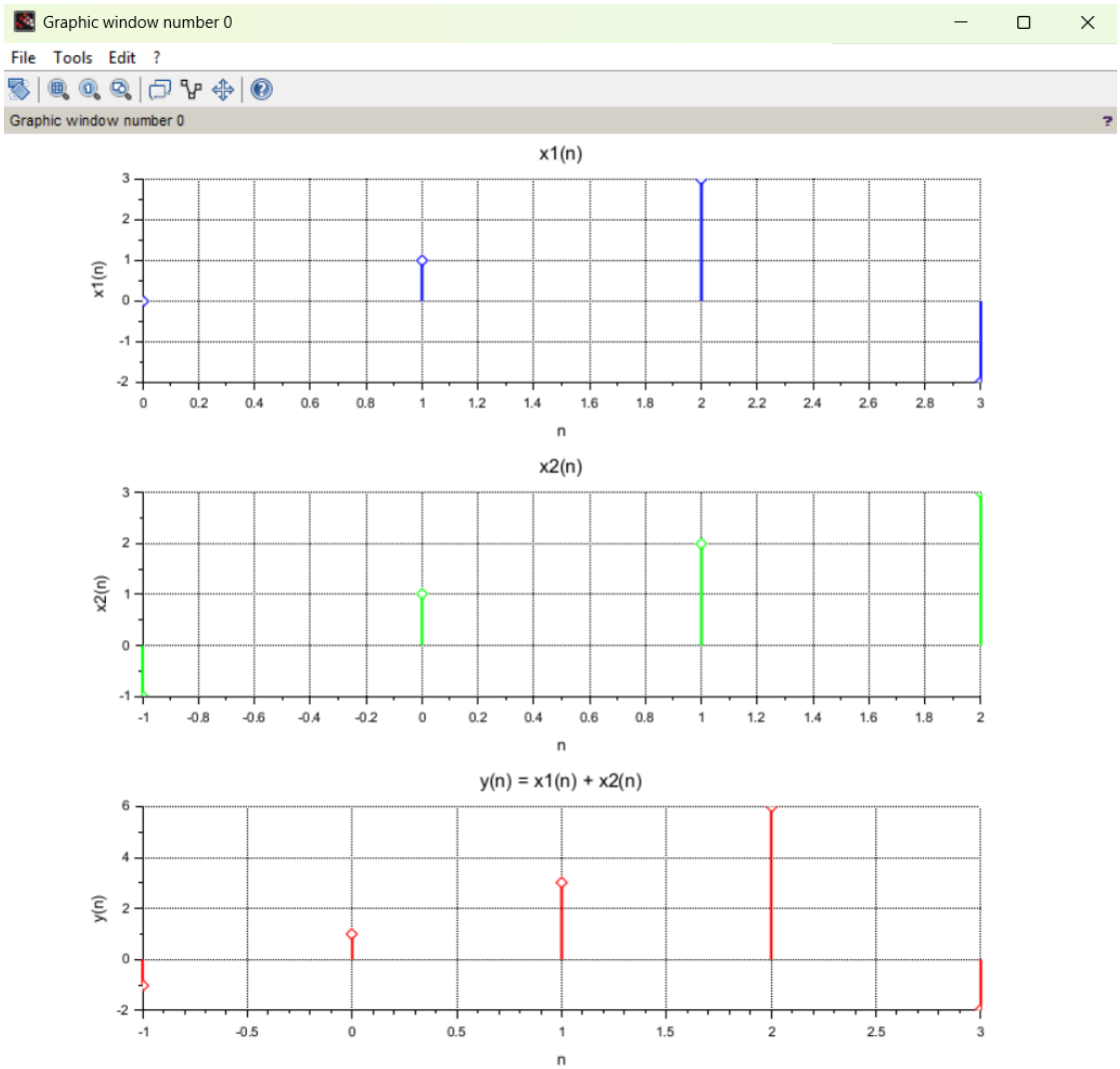
Cài đặt trên Scilab

```
function [yn, yorigin] = add(x1n, x1origin, x2n, x2origin)
    n1 = (1:length(x1n)) - x1origin;   // time axis of x1(n)
    n2 = (1:length(x2n)) - x2origin;   // time axis of x2(n)

    nmin = min(min(n1), min(n2));       // union time axis start
    nmax = max(max(n1), max(n2));       // union time axis end
    ny   = nmin:nmax;
    yorigin = 1 - nmin;

    y1 = zeros(1, length(ny));
    y2 = zeros(1, length(ny));
    y1(n1 - nmin + 1) = x1n;           // zero-pad x1(n) to common axis
    y2(n2 - nmin + 1) = x2n;           // zero-pad x2(n) to common axis
    yn = y1 + y2;                       // element-wise addition

    scf(); clf();
    // Plot x1(n)
    subplot(3,1,1);
    plot2d3(n1, x1n, style=2); gce().children.thickness = 2;
    plot2d(n1, x1n, style=-5); gce().children.mark_foreground = 2;
    xgrid(); xlabel('n'); ylabel('x1(n)'); title('x1(n)');
    // Plot x2(n)
    subplot(3,1,2);
    plot2d3(n2, x2n, style=3); gce().children.thickness = 2;
    plot2d(n2, x2n, style=-5); gce().children.mark_foreground = 3;
    xgrid(); xlabel('n'); ylabel('x2(n)'); title('x2(n)');
    // Plot y(n) = x1(n) + x2(n)
```

```
    subplot(3,1,3);
    plot2d3(ny, yn, style=5); gce().children.thickness = 2;
    plot2d(ny, yn, style=-5); gce().children.mark_foreground = 5;
    xgrid(); xlabel('n'); ylabel('y(n)'); title('y(n) = x1(n) + x2(n)');
endfunction

[yn, yorigin] = add([0,1,3,-2], 1, [-1,1,2,3], 2);
disp('yn = '); disp(yn);
disp('yorigin = '); disp(yorigin);
```

Kết quả Console và Đồ thị minh họa

```
--> exec('C:\Users\ASUS\Documents\Xử Lí Tín Hiệu Số\Lab\Lab3\Ex34.sci', -1)
 "yn ="
 -1.   1.   3.   6.  -2.
 "yorigin ="
  2.
```



Hình 4: *Đồ thị tín hiệu $x_1(n)$, $x_2(n)$ và tín hiệu tổng $y(n) = x_1(n) + x_2(n)$*

4.5 Exercise 5: Thực hiện phép nhân tín hiệu (Signal Multiplication)

Yêu cầu đề bài

- **Xây dựng hàm Scilab:**

  `function [yn, yorigin] = multi(x1n, x1origin, x2n, x2origin)`

  để thực hiện phép nhân hai tín hiệu rời rạc theo công thức:

$$y(n) = x_1(n) \cdot x_2(n)$$

- **Ý nghĩa các tham số:**
  - `x1n`: **vector biểu diễn tín hiệu** $x_1(n)$
  - `x2n`: **vector biểu diễn tín hiệu** $x_2(n)$
  - `x1origin`: **vị trí gốc của tín hiệu** $x_1(n)$
  - `x2origin`: **vị trí gốc của tín hiệu** $x_2(n)$
  - `yn`: **vector biểu diễn tín hiệu đầu ra** $y(n)$
  - `yorigin`: **vị trí gốc của tín hiệu** $y(n)$
- **Phép nhân tín hiệu được thực hiện bằng cách nhân từng phần tử tương ứng của hai tín hiệu** $x_1(n)$ **và** $x_2(n)$ **sau khi đã căn chỉnh trục thời gian của hai tín hiệu.**
- **Sau khi thực hiện phép toán, các tín hiệu** $x_1(n)$, $x_2(n)$ **và** $y(n)$ **phải được hiển thị bằng đồ thị trong cùng một figure để thuận tiện cho việc so sánh.**

Mục tiêu

Xây dựng hàm Scilab thực hiện phép toán trên tín hiệu rời rạc, được định nghĩa theo công thức:

$$y(n) = x_1(n) \cdot x_2(n)$$

Phép nhân hai tín hiệu rời rạc thực hiện nhân từng giá trị mẫu tương ứng tại cùng thời điểm n.

Đặc điểm:

- Hai tín hiệu có thể có miền xác định khác nhau, cần mở rộng về miền hợp (union) trước khi nhân.
- Các vị trí không thuộc miền xác định của tín hiệu được **zero-padding** (bổ sung giá trị 0).
- Miền xác định của tín hiệu đầu ra:

$$n_{\min} = \min(n_{\min}^{(1)},\ n_{\min}^{(2)}), \quad n_{\max} = \max(n_{\max}^{(1)},\ n_{\max}^{(2)})$$

- Gốc tọa độ đầu ra:

$$y_{\text{origin}} = 1 - n_{\min}$$

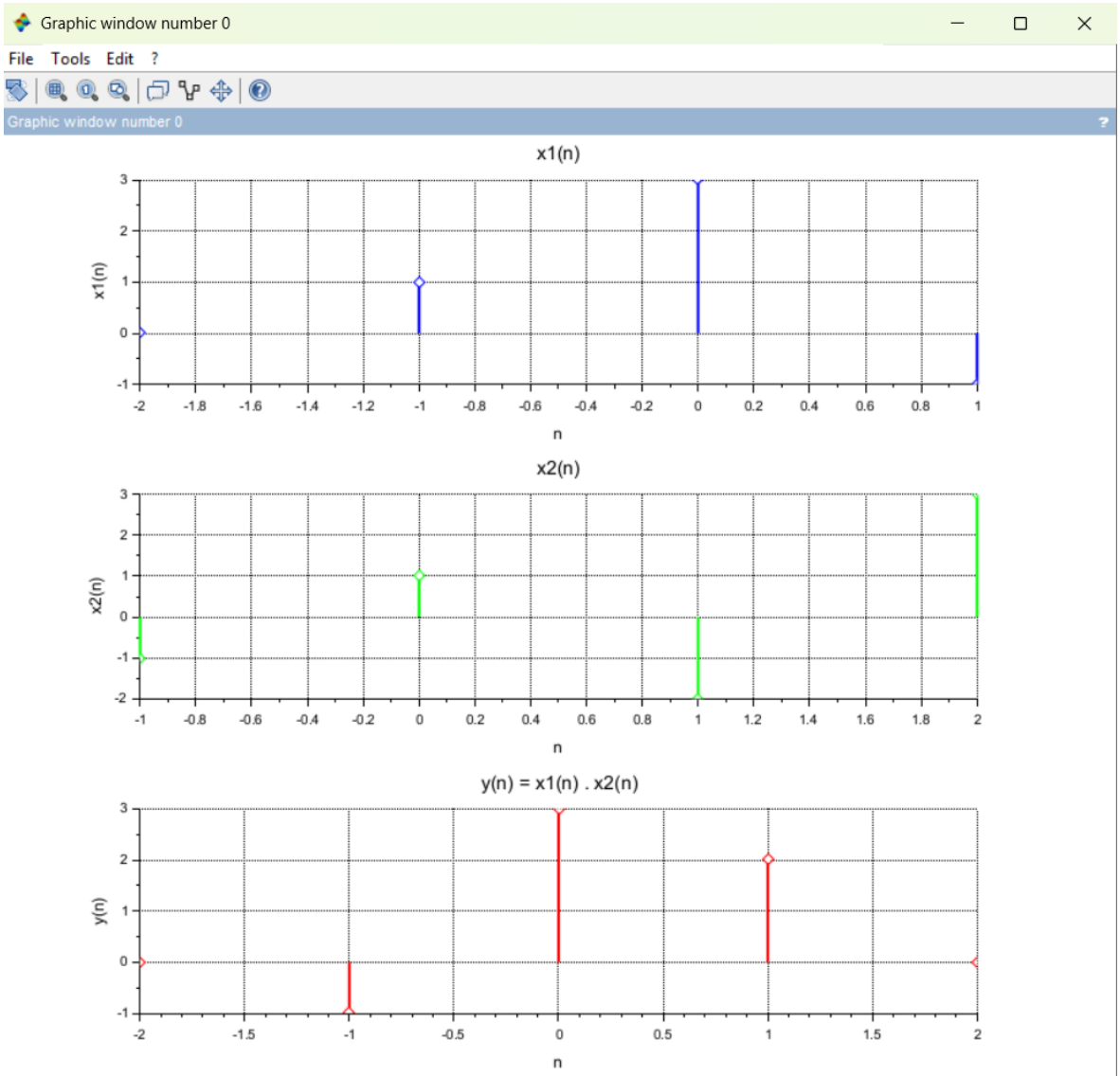
Cài đặt trên Scilab

```
// ============================================================
// FUNCTION : multi
// PURPOSE  : Perform multiplication y(n) = x1(n) . x2(n)
// INPUTS   : x1n      - sample values of signal x1(n)
//            x1origin- index of n=0 in x1n (1-based)
//            x2n      - sample values of signal x2(n)
//            x2origin- index of n=0 in x2n (1-based)
// OUTPUTS  : yn       - sample values of output signal y(n)
//            yorigin - index of n=0 in vector yn (1-based)
// ============================================================
function [yn, yorigin] = multi(x1n, x1origin, x2n, x2origin)
    n1 = (1:length(x1n)) - x1origin;   // time axis of x1(n)
    n2 = (1:length(x2n)) - x2origin;   // time axis of x2(n)

    nmin = min(min(n1), min(n2));       // union time axis start
    nmax = max(max(n1), max(n2));       // union time axis end
    ny   = nmin:nmax;
    yorigin = 1 - nmin;

    y1 = zeros(1, length(ny));
    y2 = zeros(1, length(ny));
    y1(n1 - nmin + 1) = x1n;           // zero-pad x1(n) to common axis
    y2(n2 - nmin + 1) = x2n;           // zero-pad x2(n) to common axis
    yn = y1 .* y2;                      // element-wise multiplication

    scf(); clf();

    // Plot x1(n)
    subplot(3,1,1);
    plot2d3(n1, x1n, style=2); gce().children.thickness = 2;
    plot2d(n1, x1n, style=-5); gce().children.mark_foreground = 2;
    xgrid(); xlabel('n'); ylabel('x1(n)'); title('x1(n)');

    // Plot x2(n)
    subplot(3,1,2);
    plot2d3(n2, x2n, style=3); gce().children.thickness = 2;
    plot2d(n2, x2n, style=-5); gce().children.mark_foreground = 3;
    xgrid(); xlabel('n'); ylabel('x2(n)'); title('x2(n)');
    // Plot y(n) = x1(n) . x2(n)
    subplot(3,1,3);
    plot2d3(ny, yn, style=5); gce().children.thickness = 2;
    plot2d(ny, yn, style=-5); gce().children.mark_foreground = 5;
    xgrid(); xlabel('n'); ylabel('y(n)'); title('y(n) = x1(n) . x2(n)');
endfunction
```

```
// ------------------------------------------------------------
// Example: x1(n) = {0, 1, 3^, -1}, x1origin = 3
//          x2(n) = {-1, 1^, -2, 3}, x2origin = 2
// Result : yn = [0, -1, 3, 2, 0],  yorigin = 3
// ------------------------------------------------------------
[yn, yorigin] = multi([0,1,3,-1], 3, [-1,1,-2,3], 2);
disp('yn␣='); disp(yn);
disp('yorigin␣='); disp(yorigin);
```

Kết quả Console và Đồ thị minh họa

```
--> exec('C:\Users\ASUS\Documents\Xử Lí Tín Hiệu Số\Lab\Lab3\Ex35.sci', -1)
 "yn ="
  0.  -1.   3.   2.   0.
 "yorigin ="
  3.
```



Hình 5: *Đồ thị tín hiệu $x_1(n)$, $x_2(n)$ và tín hiệu tích $y(n) = x_1(n) \cdot x_2(n)$*

4.6 Exercise 6: Thực hiện phép tích chập (Convolution)

Yêu cầu đề bài

- **Xây dựng hàm Scilab:**

  `function [yn, yorigin] = convolution(xn, xorigin, hn, horigin)`

  để thực hiện phép tích chập của hai tín hiệu rời rạc theo công thức:

$$y(n) = x(n) * h(n)$$

$$y(n) = \sum_{k=-\infty}^{\infty} x(k)\, h(n-k)$$

- **Ý nghĩa các tham số:**
  - `xn`: **vector biểu diễn tín hiệu đầu vào** $x(n)$
  - `hn`: **vector biểu diễn hàm đặc trưng của hệ thống** $h(n)$
  - `xorigin`: **vị trí gốc của tín hiệu** $x(n)$
  - `horigin`: **vị trí gốc của tín hiệu** $h(n)$
  - `yn`: **vector biểu diễn tín hiệu đầu ra** $y(n)$
  - `yorigin`: **vị trí gốc của tín hiệu** $y(n)$

- **Phép tích chập mô tả mối quan hệ giữa tín hiệu đầu vào và đáp ứng của hệ thống trong các hệ thống xử lý tín hiệu số. Kết quả của phép tích chập cho biết tín hiệu đầu ra của hệ thống khi tín hiệu** $x(n)$ **đi qua hệ thống có đặc tính** $h(n)$**.**

- **Sau khi thực hiện phép toán, các tín hiệu** $x(n)$**,** $h(n)$ **và** $y(n)$ **phải được hiển thị bằng đồ thị trong cùng một figure để minh họa kết quả của phép tích chập.**

Mục tiêu

Xây dựng hàm Scilab thực hiện phép toán trên tín hiệu rời rạc, được định nghĩa theo công thức:

$$y(n) = x(n) * h(n) = \sum_{k=-\infty}^{+\infty} x(k) \cdot h(n-k)$$

Phép tích chập là phép toán cơ bản trong xử lý tín hiệu số, dùng để xác định tín hiệu đầu ra của một hệ thống tuyến tính bất biến (LTI) khi biết tín hiệu vào và đáp ứng xung.

Đặc điểm:

- Độ dài tín hiệu đầu ra:

$$N_y = N_x + N_h - 1$$

- Gốc tọa độ đầu ra:

$$y_{\text{origin}} = x_{\text{origin}} + h_{\text{origin}} - 1$$

- Sử dụng hàm `conv()` có sẵn trong Scilab để thực hiện phép tích chập.

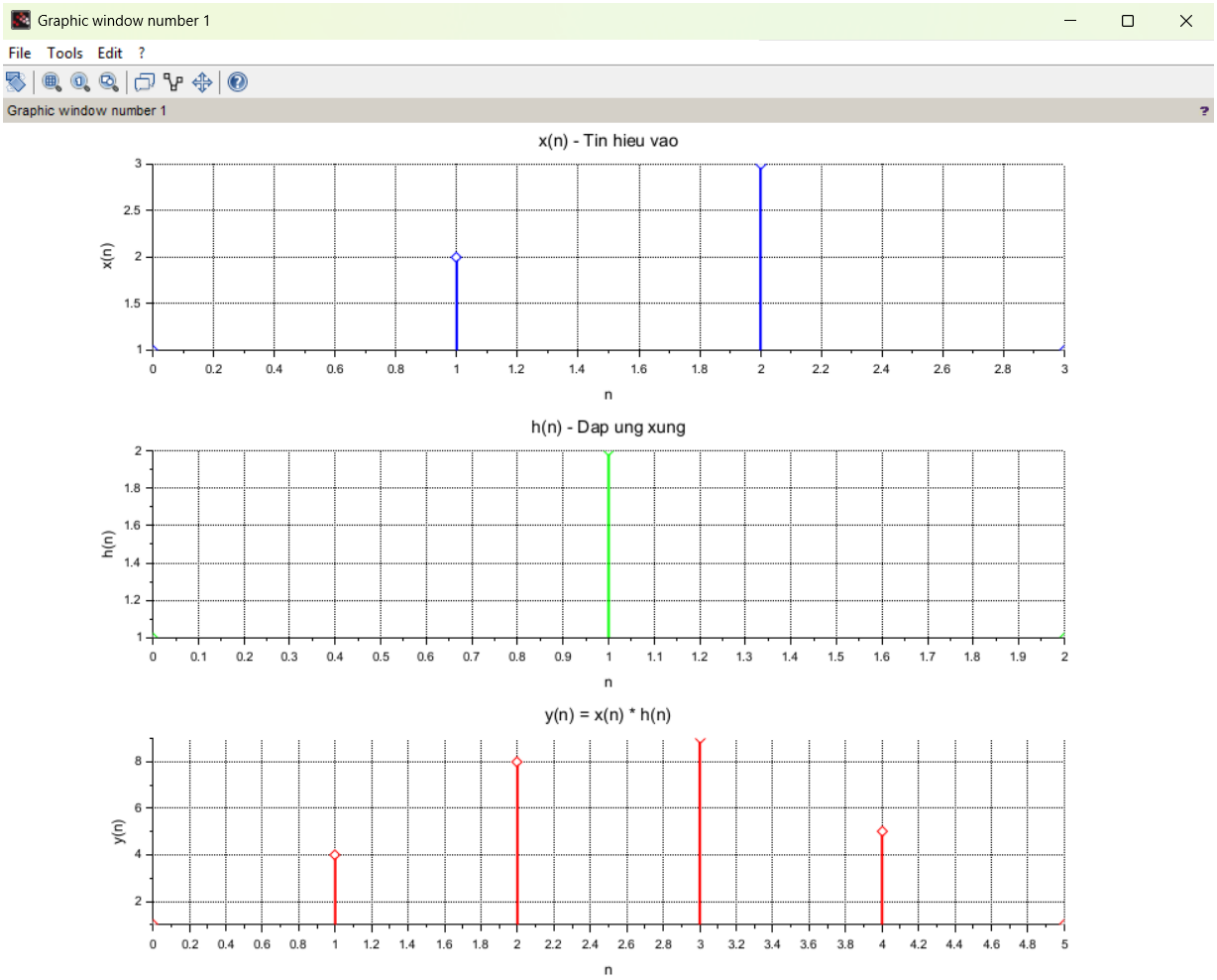
Cài đặt trên Scilab

```
// ============================================================
// FUNCTION : convolution
// PURPOSE  : Perform linear convolution y(n) = x(n) * h(n)
// INPUTS   : xn       - sample values of input signal x(n)
//            xorigin  - index of n=0 in vector xn (1-based)
//            hn       - sample values of impulse response h(n)
//            horigin  - index of n=0 in vector hn (1-based)
// OUTPUTS  : yn       - sample values of output signal y(n)
//            yorigin  - index of n=0 in vector yn (1-based)
// NOTE     : Output length = length(xn) + length(hn) - 1
// ============================================================
function [yn, yorigin] = convolution(xn, xorigin, hn, horigin)
    Nx = length(xn);
    Nh = length(hn);

    yn = conv(xn, hn);                          // built-in convolution

    ny_start = (1 - xorigin) + (1 - horigin);  // first n-index of y(n)
    ny = ny_start : (ny_start + Nx + Nh - 2);  // time axis of y(n)
    yorigin = xorigin + horigin - 1;            // origin of y(n)

    nx = (1:Nx) - xorigin;                      // time axis of x(n)
    nh = (1:Nh) - horigin;                      // time axis of h(n)

    scf(); clf();
    // Plot x(n) - input signal
    subplot(3,1,1);
    plot2d3(nx, xn, style=2); gce().children.thickness = 2;
    plot2d(nx, xn, style=-5); gce().children.mark_foreground = 2;
    xgrid(); xlabel('n'); ylabel('x(n)'); title('x(n) - Tin hieu vao');

    // Plot h(n) - impulse response
    subplot(3,1,2);
    plot2d3(nh, hn, style=3); gce().children.thickness = 2;
    plot2d(nh, hn, style=-5); gce().children.mark_foreground = 3;
    xgrid(); xlabel('n'); ylabel('h(n)'); title('h(n) - Dap ung xung');

    // Plot y(n) = x(n) * h(n)
    subplot(3,1,3);
    plot2d3(ny, yn, style=5); gce().children.thickness = 2;
    plot2d(ny, yn, style=-5); gce().children.mark_foreground = 5;
    xgrid(); xlabel('n'); ylabel('y(n)'); title('y(n) = x(n) * h(n)');
endfunction

```

```
// ------------------------------------------------------------
// Example: x(n) = {1^, 2, 3, 1}, xorigin = 1
//          h(n) = {1^, 2, 1},    horigin = 1
// Result : yn = [1, 4, 8, 8, 5, 1],  yorigin = 1
// ------------------------------------------------------------
[yn, yorigin] = convolution([1,2,3,1], 1, [1,2,1], 1);
disp('yn␣='); disp(yn);
disp('yorigin␣='); disp(yorigin);
```

Kết quả Console và Đồ thị minh họa

```
--> exec('C:\Users\ASUS\Documents\Xử Lí Tín Hiệu Số\Lab\Lab3\Ex36.sci', -1)
 "yn ="
  1.   4.   8.   9.   5.   1.
 "yorigin ="
  1.
```



Hình 6: *Đồ thị tín hiệu vào $x(n)$, đáp ứng xung $h(n)$ và tín hiệu ra $y(n) = x(n) * h(n)$*

5 Giải các bài tập tín hiệu rời rạc trong tài liệu tham khảo

5.1 Problem 2.1: Phân tích tín hiệu rời rạc

Yêu cầu đề bài

- **Một tín hiệu rời rạc $x(n)$ được định nghĩa như sau:**

$$x(n) = \begin{cases} 1 + \frac{n}{3}, & -3 \le n \le -1 \\ 1, & 0 \le n \le 3 \\ 0, & \textbf{otherwise} \end{cases}$$

- **Thực hiện các yêu cầu sau:**
  - (a) **Xác định các giá trị của tín hiệu và vẽ đồ thị biểu diễn tín hiệu $x(n)$.**
  - (b) **Vẽ các tín hiệu thu được khi thực hiện các phép biến đổi sau:**
    - ∗ **Fold tin hiệu $x(n)$ rồi sau đó Delay tín hiệu kết quả đi 4 mẫu.**
    - ∗ **Delay tín hiệu $x(n)$ đi 4 mẫu rồi sau đó Fold tín hiệu kết quả.**
  - (c) **Vẽ tín hiệu $x(-n+4)$.**
  - (d) **So sánh kết quả thu được ở các phần (b) và (c), từ đó rút ra quy tắc để xác định tín hiệu $x(-n+k)$ từ tín hiệu $x(n)$.**
  - (e) **Biểu diễn tín hiệu $x(n)$ theo các tín hiệu cơ bản $\delta(n)$ (unit impulse) và $u(n)$ (unit step).**

(a) Xác định giá trị và vẽ đồ thị $x(n)$

Tín hiệu rời rạc $x(n)$ được định nghĩa:

$$x(n) = \begin{cases} 1 + \dfrac{n}{3}, & -3 \le n \le -1 \\ 1, & 0 \le n \le 3 \\ 0, & \text{otherwise} \end{cases}$$

Tính giá trị $x(n)$ tại từng điểm:

- Vùng $-3 \le n \le -1$: áp dụng $x(n) = 1 + \dfrac{n}{3}$

$$n = -3 \implies x(-3) = 1 + \frac{-3}{3} = 1 - 1 = 0$$

$$n = -2 \implies x(-2) = 1 + \frac{-2}{3} = \frac{1}{3} \approx 0{,}333$$

$$n = -1 \implies x(-1) = 1 + \frac{-1}{3} = \frac{2}{3} \approx 0{,}667$$

- Vùng $0 \leq n \leq 3$: áp dụng $x(n) = 1$

$$\begin{aligned} n = 0 &\implies x(0) = 1 \\ n = 1 &\implies x(1) = 1 \\ n = 2 &\implies x(2) = 1 \\ n = 3 &\implies x(3) = 1 \end{aligned}$$

- Ngoài hai vùng trên: $x(n) = 0$

Bảng tổng hợp giá trị $x(n)$:

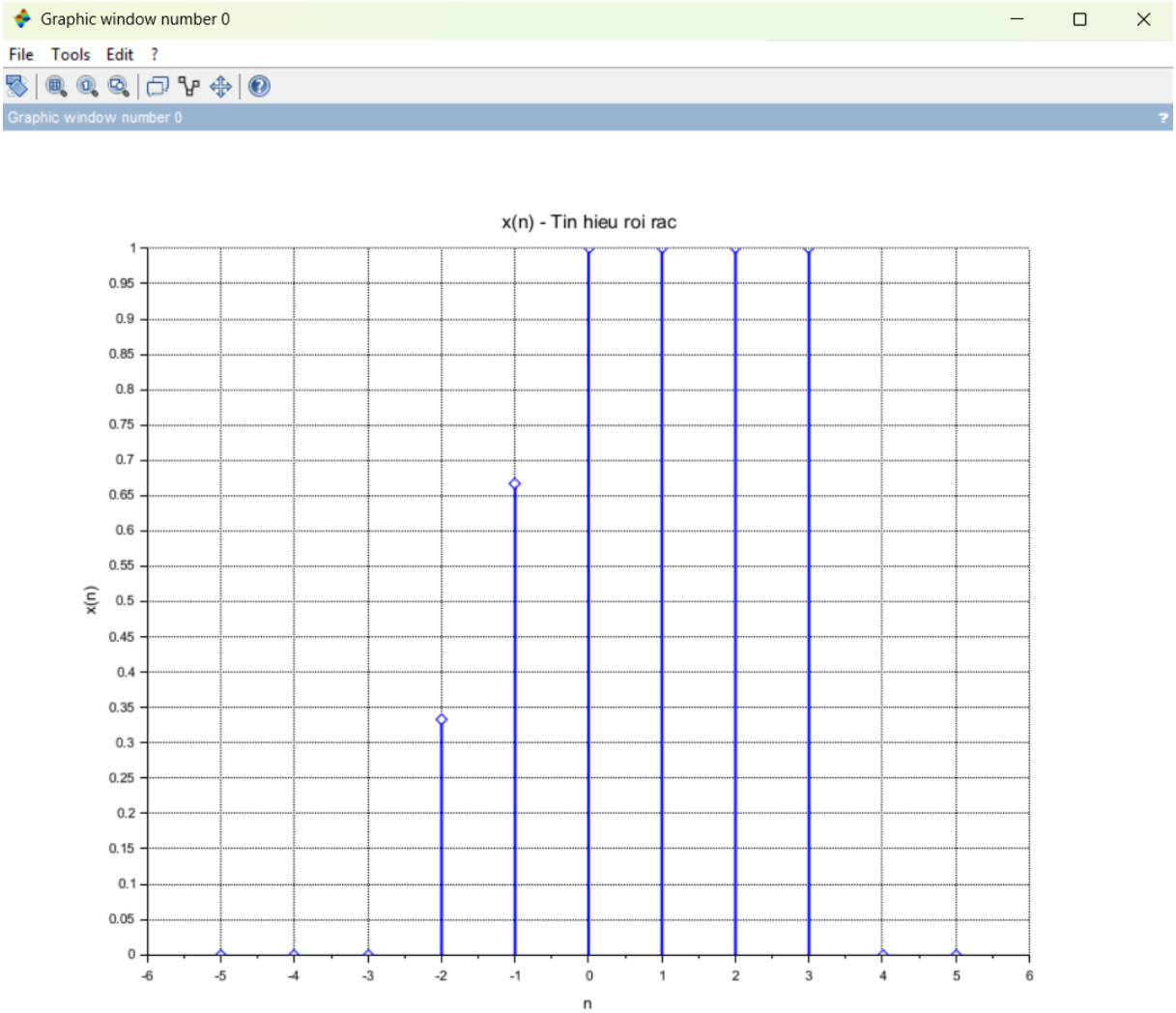
n	$\cdots$	-4	-3	-2	-1	0	1	2	3	4	$\cdots$
$x(n)$	0	0	0	$\frac{1}{3}$	$\frac{2}{3}$	1	1	1	1	0	0
Vùng		0	0	$1 + \frac{n}{3}$			1				0

Bảng 2: *Bảng giá trị tín hiệu $x(n)$ trên toàn miền xác định*

Nhận xét về dạng tín hiệu:

- Vùng $[-3,\ -1]$: tăng tuyến tính từ 0 đến $\frac{2}{3}$, đây là **dạng ramp rời rạc**.
- Vùng $[0,\ 3]$: giữ nguyên giá trị 1, đây là **dạng hằng số** (tương tự hàm bước).

```
// ==============================================================
// Problem 2.1 (a): Define and plot x(n)
// ==============================================================
n  = -5:5;
xn = zeros(1, length(n));

// Compute x(n) over each region
for i = 1:length(n)
    if n(i) >= -3 & n(i) <= -1 then
        xn(i) = 1 + n(i)/3;         // ramp region
    elseif n(i) >= 0 & n(i) <= 3 then
        xn(i) = 1;                  // flat region
    else
        xn(i) = 0;                  // zero elsewhere
    end
end

// Plot x(n)
scf(); clf();
plot2d3(n, xn, style=2); gce().children.thickness = 2;
plot2d(n, xn, style=-5); gce().children.mark_foreground = 2;
xgrid(); xlabel('n'); ylabel('x(n)'); title('x(n) - Tin hieu roi rac');
```



Hình 7: *Đồ thị tín hiệu rời rạc $x(n)$*

(b) Các phép biến đổi Fold và Delay

B1 - Fold $x(n)$ rồi Delay 4 mẫu

Thực hiện lần lượt hai bước biến đổi:

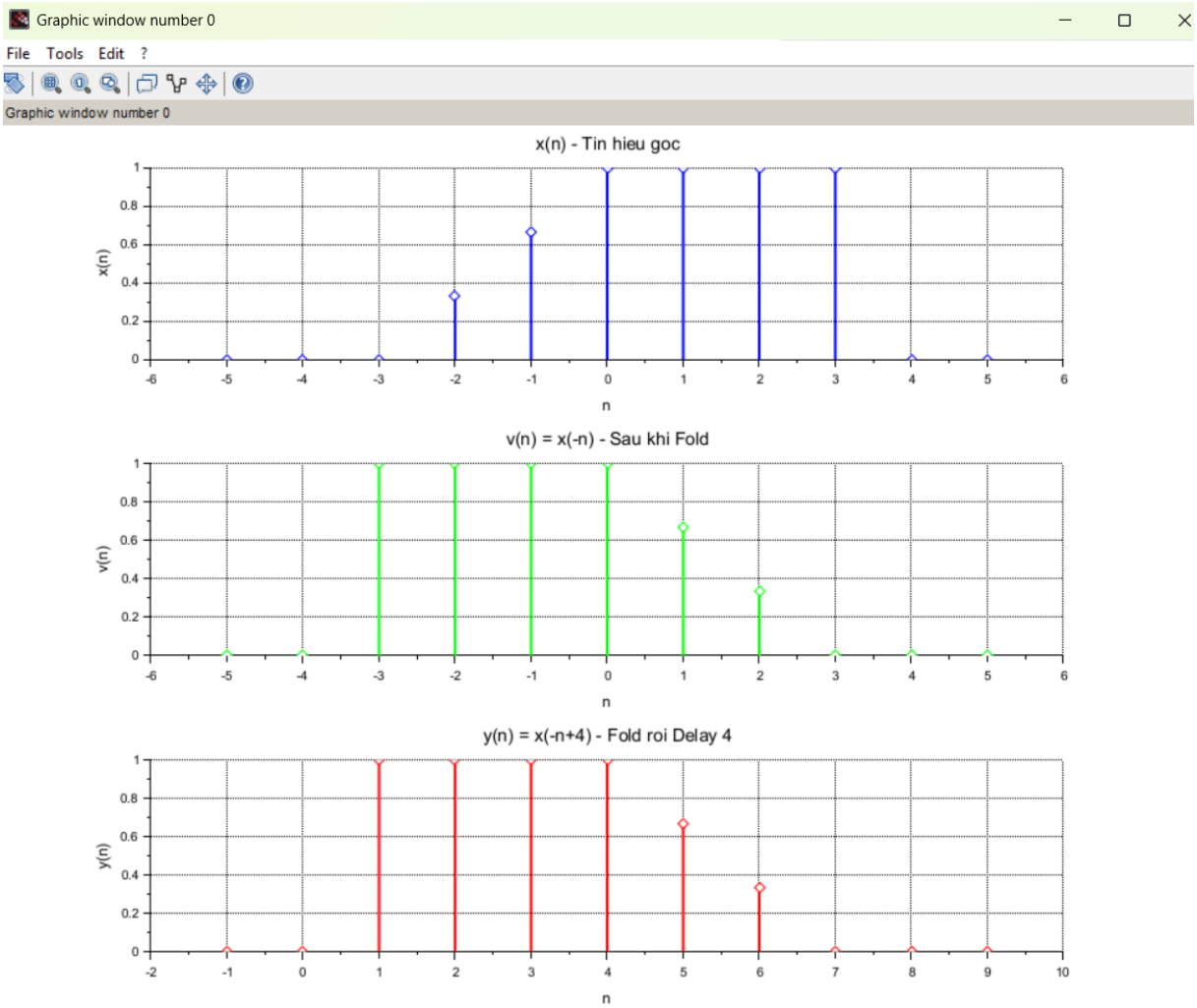
- **Bước 1 - Fold:** $v(n) = x(-n)$
  Lật ngược tín hiệu $x(n)$ quanh trục $n = 0$, gốc tọa độ được tính lại:

$$v_{\text{origin}} = N - x_{\text{origin}} + 1$$

- **Bước 2 - Delay 4 mẫu:** $y(n) = v(n-4)$
  Dịch chuyển tín hiệu $v(n)$ sang phải 4 bước, kết hợp hai bước:

$$y(n) = v(n-4) = x\big(-(n-4)\big) = x(-n+4)$$

```
// ============================================================
// Problem 2.1 (b1): Fold x(n) then Delay by 4
// Step 1: v(n) = x(-n)
// Step 2: y(n) = v(n-4) = x(-n+4)
// ============================================================
xorigin = 6;                              // n=0 is at index 6 (n=-5..5)

// Step 1: Fold
vn      = xn($:-1:1);                    // reverse sample vector
vorigin = length(xn) - xorigin + 1;    // mirror origin position

// Step 2: Delay by 4
yn_b1      = vn;
yorigin_b1 = vorigin - 4;              // delay shifts origin left by 4

nv    = (1:length(vn))    - vorigin;
ny_b1 = (1:length(yn_b1)) - yorigin_b1;

// Plot
scf(); clf();
subplot(3,1,1);
plot2d3(n, xn, style=2); gce().children.thickness = 2;
plot2d(n, xn, style=-5); gce().children.mark_foreground = 2;
xgrid(); xlabel('n'); ylabel('x(n)'); title('x(n) - Tin hieu goc');

subplot(3,1,2);
plot2d3(nv, vn, style=3); gce().children.thickness = 2;
plot2d(nv, vn, style=-5); gce().children.mark_foreground = 3;
xgrid(); xlabel('n'); ylabel('v(n)'); title('v(n) = x(-n) - Sau khi Fold');

subplot(3,1,3);
plot2d3(ny_b1, yn_b1, style=5); gce().children.thickness = 2;
plot2d(ny_b1, yn_b1, style=-5); gce().children.mark_foreground = 5;
xgrid(); xlabel('n'); ylabel('y(n)'); title('y(n) = x(-n+4) - Fold roi Delay 4');
```



Hình 8: *Kết quả Fold $x(n)$ rồi Delay 4 mẫu: $y(n) = x(-n+4)$*

B2 - Delay $x(n)$ 4 mẫu rồi Fold

Thực hiện lần lượt hai bước biến đổi:

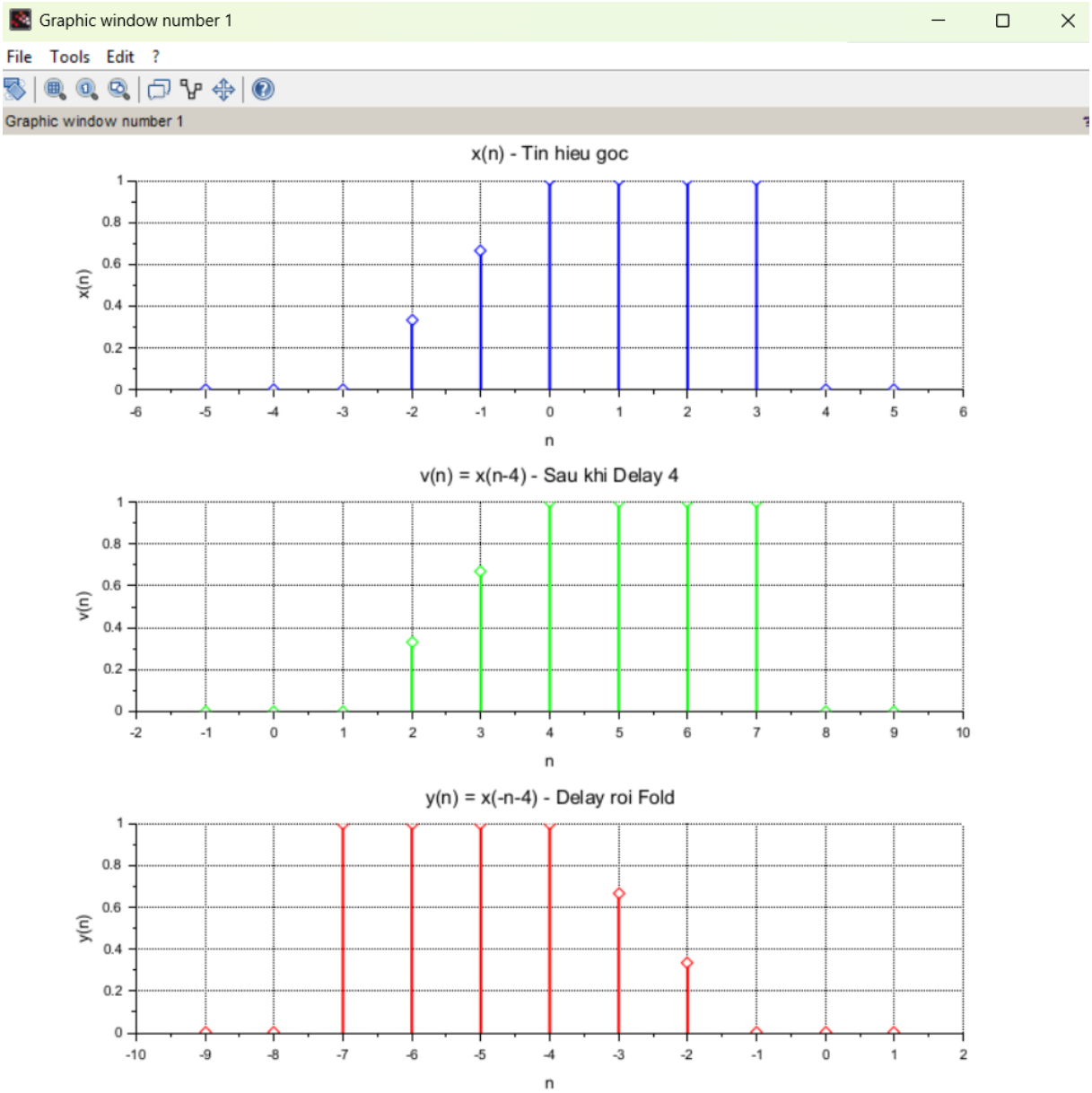
- **Bước 1 - Delay 4 mẫu:** $v(n) = x(n-4)$
  Dịch chuyển tín hiệu $x(n)$ sang phải 4 bước, gốc tọa độ dịch chuyển:

$$v_{\text{origin}} = x_{\text{origin}} - 4$$

- **Bước 2 - Fold:** $y(n) = v(-n)$
  Lật ngược tín hiệu $v(n)$ quanh trục $n = 0$, kết hợp hai bước:

$$y(n) = v(-n) = x(-n-4)$$

```
// ============================================================
// Problem 2.1 (b2): Delay x(n) by 4 then Fold
// Step 1: v(n) = x(n-4)
// Step 2: y(n) = v(-n) = x(-n-4)
// ============================================================
xorigin  = 6;

// Step 1: Delay by 4
vn2      = xn;
vorigin2 = xorigin - 4;                // delay shifts origin left by 4

// Step 2: Fold
yn_b2      = vn2($:-1:1);              // reverse sample vector
yorigin_b2 = length(vn2) - vorigin2 + 1;

nv2   = (1:length(vn2))   - vorigin2;
ny_b2 = (1:length(yn_b2)) - yorigin_b2;

// Plot
scf(); clf();
subplot(3,1,1);
plot2d3(n, xn, style=2); gce().children.thickness = 2;
plot2d(n, xn, style=-5); gce().children.mark_foreground = 2;
xgrid(); xlabel('n'); ylabel('x(n)'); title('x(n) - Tin hieu goc');

subplot(3,1,2);
plot2d3(nv2, vn2, style=3); gce().children.thickness = 2;
plot2d(nv2, vn2, style=-5); gce().children.mark_foreground = 3;
xgrid(); xlabel('n'); ylabel('v(n)'); title('v(n) = x(n-4) - Sau khi Delay 4');

subplot(3,1,3);
plot2d3(ny_b2, yn_b2, style=5); gce().children.thickness = 2;
plot2d(ny_b2, yn_b2, style=-5); gce().children.mark_foreground = 5;
xgrid(); xlabel('n'); ylabel('y(n)'); title('y(n) = x(-n-4) - Delay roi Fold');
```



Hình 9: *Kết quả Delay 4 mẫu rồi Fold $x(n)$: $y(n) = x(-n-4)$*

(c) Vẽ tín hiệu $x(-n+4)$

Thay trực tiếp $m = -n + 4$ vào công thức định nghĩa của $x(n)$:

$$x(-n+4) = \begin{cases} 1 + \dfrac{-n+4}{3}, & -3 \leq -n+4 \leq -1 \\ 1, & 0 \leq -n+4 \leq 3 \\ 0, & \text{otherwise} \end{cases}$$

Giải các bất phương trình để tìm miền xác định theo n:

$$-3 \le -n+4 \le -1 \implies 5 \le n \le 7$$
$$0 \le -n+4 \le 3 \implies 1 \le n \le 4$$

Vậy:

$$x(-n+4) = \begin{cases} 1 + \dfrac{-n+4}{3}, & 5 \le n \le 7 \\ 1, & 1 \le n \le 4 \\ 0, & \text{otherwise} \end{cases}$$

Bảng giá trị $x(-n+4)$:

n	1	2	3	4	5	6	7
$m = -n+4$	3	2	1	0	-1	-2	-3
$x(-n+4)$	1	1	1	1	$\frac{2}{3}$	$\frac{1}{3}$	0

Bảng 3: *Bảng giá trị tín hiệu $x(-n+4)$*

Nhận xét:

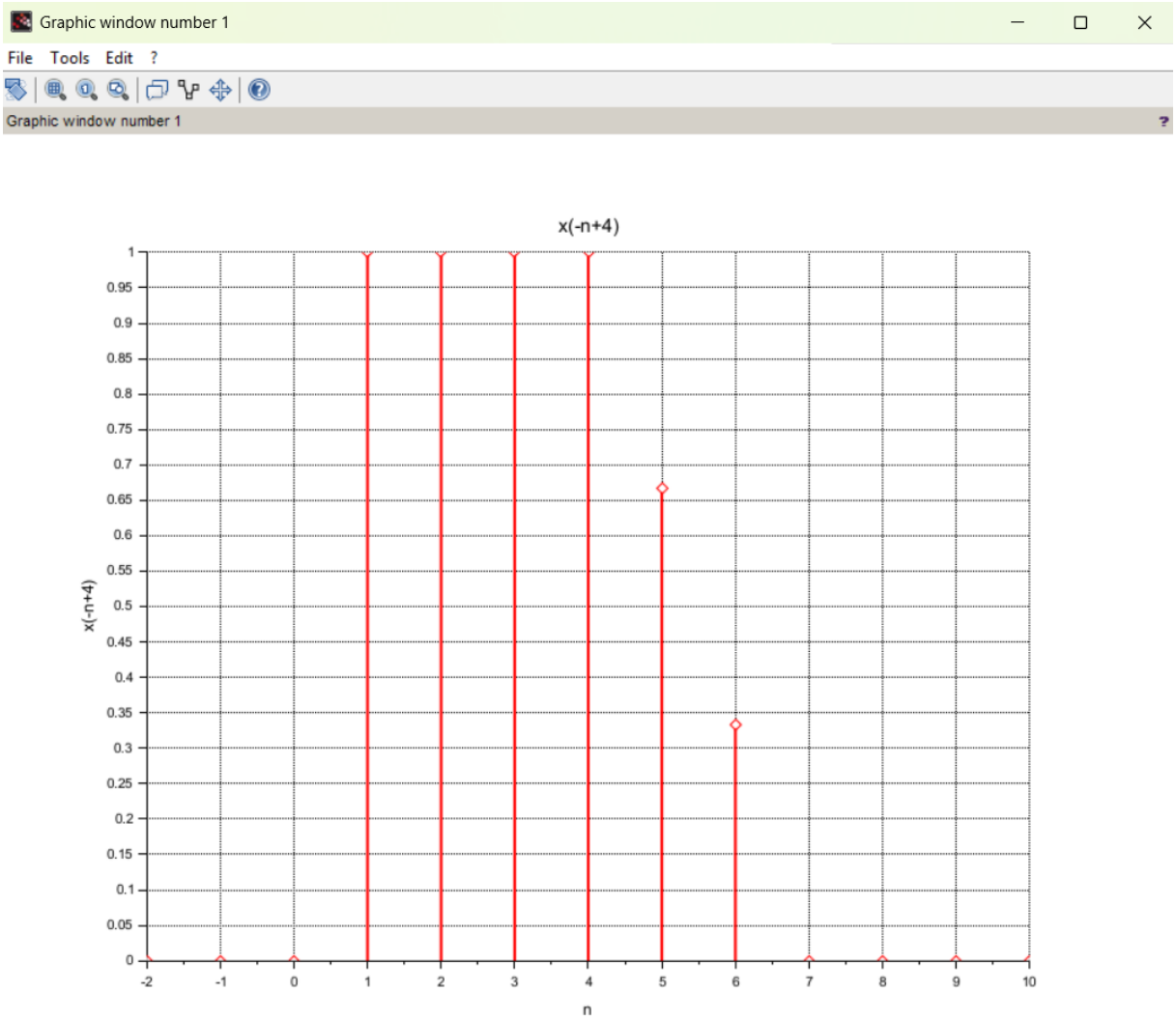
- Tín hiệu $x(-n+4)$ xác định trên đoạn $[1,\ 7]$, dịch chuyển hoàn toàn sang **miền dương** so với $x(n)$ xác định trên $[-3,\ 3]$.
- So sánh với kết quả phần **(b1)** - Fold rồi Delay 4 mẫu:

  $$y(n) = x(-n+4)$$

  Hai kết quả **hoàn toàn trùng khớp**, xác nhận tính đúng đắn của phép tính.
- Các giá trị mẫu **giảm dần** từ $n = 1$ đến $n = 7$:
  - Vùng $1 \le n \le 4$: giá trị bằng 1 (hằng số).
  - Vùng $5 \le n \le 7$: giá trị giảm tuyến tính từ $\frac{2}{3}$ xuống 0.
- Đây chính là tín hiệu $x(n)$ sau khi được **Fold** và **Delay 4 đơn vị** trên trục n.

```
// ============================================================
// Problem 2.1 (c): Plot x(-n+4) directly
// Substitute m = -n+4 into the definition of x(n)
// ============================================================
n_c  = -5:5;
xn_c = zeros(1, length(n_c));

for i = 1:length(n_c)
    m = -n_c(i) + 4;                    // substitute m = -n+4
    if m >= -3 & m <= -1 then
```

```
        xn_c(i) = 1 + m/3;
    elseif m >= 0 & m <= 3 then
        xn_c(i) = 1;
    else
        xn_c(i) = 0;
    end
end

// Plot x(-n+4)
scf(); clf();
plot2d3(n_c, xn_c, style=5); gce().children.thickness = 2;
plot2d(n_c, xn_c, style=-5); gce().children.mark_foreground = 5;
xgrid(); xlabel('n'); ylabel('x(-n+4)'); title('x(-n+4)');
```



Hình 10: *Đồ thị tín hiệu* $x(-n+4)$

(d) So sánh và rút ra quy tắc

Phép biến đổi	**Kết quả**
Fold rồi Delay 4 mẫu	$x(-n+4)$
Delay 4 mẫu rồi Fold	$x(-n-4)$
Tính trực tiếp - phần (c)	$x(-n+4)$

Bảng 4: *So sánh kết quả các phép biến đổi*

Quy tắc tổng quát để xác định $x(-n+k)$ từ $x(n)$:

- **Fold trước, Delay k mẫu sau:**

$$v(n) = x(-n) \xrightarrow{\text{Delay } k} y(n) = v(n-k) = x(-(n-k)) = x(-n+k)$$

- **Delay k mẫu trước, Fold sau:**

$$v(n) = x(n-k) \xrightarrow{\text{Fold}} y(n) = v(-n) = x(-n-k)$$

Quy tắc

Để thu được $x(-n+k)$ từ $x(n)$, cần thực hiện **Fold trước** rồi **Delay k mẫu sau**. Thứ tự ngược lại cho kết quả $x(-n-k)$, khác dấu với k.

(e) Biểu diễn $x(n)$ theo $\delta(n)$ và $u(n)$

Xung đơn vị $\delta(n)$ (Unit Impulse) được định nghĩa:

$$\delta(n) = \begin{cases} 1, & n = 0 \\ 0, & n \neq 0 \end{cases}$$

Bất kỳ tín hiệu rời rạc nào cũng có thể biểu diễn dưới dạng tổng các xung đơn vị dịch chuyển:

$$x(n) = \sum_{k=-\infty}^{+\infty} x(k)\,\delta(n-k)$$

Hàm bước đơn vị $u(n)$ (Unit Step) được định nghĩa:

$$u(n) = \begin{cases} 1, & n \geq 0 \\ 0, & n < 0 \end{cases}$$

Mối quan hệ giữa $\delta(n)$ và $u(n)$:

$$u(n) = \sum_{k=0}^{+\infty} \delta(n-k), \qquad \delta(n) = u(n) - u(n-1)$$

Hàm bước đơn vị dịch chuyển $u(n-n_0)$ bằng 1 với mọi $n \geq n_0$ và bằng 0 với $n < n_0$, do đó hiệu $u(n-a) - u(n-b)$ tạo ra một **cửa sổ chữ nhật** trên đoạn $[a,\ b-1]$.

1. Biểu diễn qua xung đơn vị $\delta(n)$

Từ bảng giá trị đã tính ở phần (a), tín hiệu $x(n)$ có dạng:

$$x(n) = \{\ldots,\ 0,\ 0,\ \tfrac{1}{3},\ \tfrac{2}{3},\ \underset{\uparrow}{1},\ 1,\ 1,\ 1,\ 0,\ \ldots\}$$

Áp dụng tính chất khai triển chuỗi xung:

$$x(n) = \sum_{k=-\infty}^{+\infty} x(k)\,\delta(n-k)$$

Vì $x(n) = 0$ ngoài $[-2,\ 3]$, chỉ giữ lại các số hạng khác 0:

$$\boxed{x(n) = \frac{1}{3}\,\delta(n+2) + \frac{2}{3}\,\delta(n+1) + \delta(n) + \delta(n-1) + \delta(n-2) + \delta(n-3)}$$

2. Biểu diễn qua hàm bước đơn vị $u(n)$

Ta có tính chất liên hệ giữa tín hiệu xung đơn vị $\delta(n)$ và tín hiệu bước đơn vị $u(n)$:

$$\delta(n) = u(n) - u(n-1)$$

Vì vậy, có thể biến đổi biểu thức của $x(n)$ theo $u(n)$ như sau:

$$x(n) = \frac{1}{3}\big(u(n+2) - u(n+1)\big) + \frac{2}{3}\big(u(n+1) - u(n)\big) + \big(u(n) - u(n-1)\big)$$

$$+\big(u(n-1) - u(n-2)\big) + \big(u(n-2) - u(n-3)\big) + \big(u(n-3) - u(n-4)\big)$$

Rút gọn các hạng tử ta được:

$$\boxed{x(n) = \frac{1}{3}u(n+2) + \frac{1}{3}u(n+1) + \frac{1}{3}u(n) - u(n-4)}$$

5.2 Problem 2.2: Biến đổi tín hiệu rời rạc

Yêu cầu đề bài

- **Một tín hiệu rời rạc $x(n)$ được cho trong Hình P2.2.**

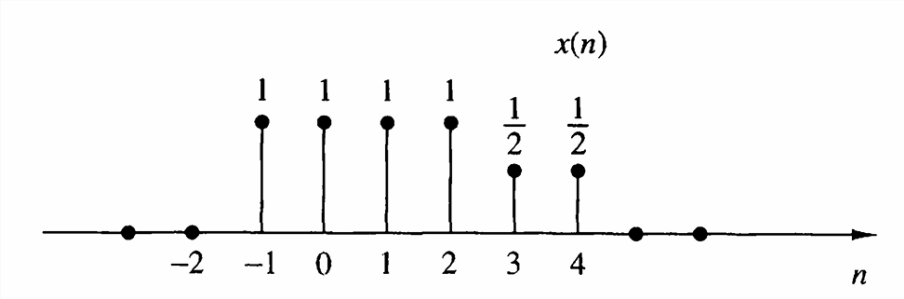


Figure P2.2. *Discrete-time signal $x(n)$*

- **Hãy vẽ và ghi chú cẩn thận các tín hiệu sau:**
  - **(a)** $x(n-2)$
  - **(b)** $x(4-n)$
  - **(c)** $x(n+2)$
  - **(d)** $x(n)u(2-n)$
  - **(e)** $x(n-1)\delta(n-3)$
  - **(f)** $x(n^2)$
  - **(g) Thành phần chẵn của** $x(n)$
  - **(h) Thành phần lẻ của** $x(n)$
- **Các tín hiệu phải được biểu diễn bằng đồ thị rời rạc theo trục thời gian n và ghi rõ các giá trị biên độ.**

Xác định tín hiệu $x(n)$ từ đồ thị

Từ đồ thị Hình P2.2, đọc giá trị tín hiệu $x(n)$:

$$x(n) = \{\ldots,\ 0,\ 1,\ \overset{\uparrow}{1},\ 1,\ 1,\ \tfrac{1}{2},\ \tfrac{1}{2},\ 0,\ \ldots\}$$

trong đó $\uparrow$ chỉ vị trí $n = 0$.

n	$\cdots$	-2	-1	0	1	2	3	4	$\cdots$
$x(n)$	0	0	1	1	1	1	$\frac{1}{2}$	$\frac{1}{2}$	0

Bảng 5: *Giá trị tín hiệu $x(n)$ đọc từ đồ thị*

(a) $x(n-2)$ - Delay 2 mẫu

Phép trễ 2 mẫu dịch chuyển toàn bộ tín hiệu sang **phải 2 bước** trên trục n, các giá trị mẫu giữ nguyên, chỉ trục thời gian thay đổi:

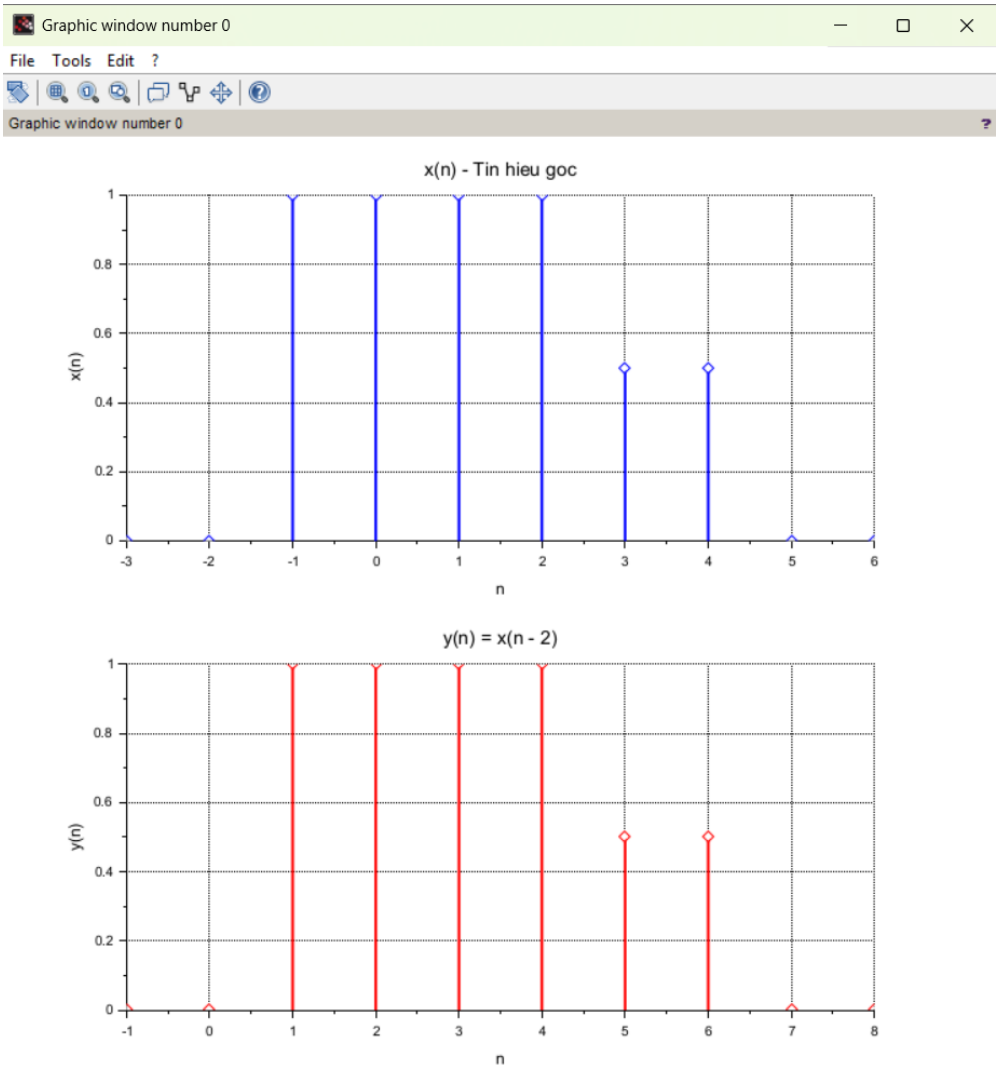
$$y(n) = x(n-2) \implies \text{tín hiệu xác định trên } [1,\ 6]$$

Bảng giá trị:

n	$\cdots$	0	1	2	3	4	5	6	$\cdots$
$x(n-2)$	0	0	1	1	1	1	$\frac{1}{2}$	$\frac{1}{2}$	0

Bảng 6: *Giá trị tín hiệu $x(n-2)$*

```
// Define signal x(n) from Figure P2.2
xn      = [0, 0, 1, 1, 1, 1, 0.5, 0.5, 0, 0];  // sample values
xorigin = 4;                                    // n=0 is at index 4
// FUNCTION : delay
function [yn, yorigin] = delay(xn, xorigin, k)
    yn      = xn;                   // sample values unchanged
    yorigin = xorigin - k;          // origin index shifts left by k
    Nx = length(xn);
    nx = (1:Nx) - xorigin;          // time axis of x(n)
    ny = (1:Nx) - yorigin;          // time axis of y(n)
    scf(); clf();

    // Plot x(n) - original signal
    subplot(2,1,1);
    plot2d3(nx, xn, style=2); gce().children.thickness = 2;
    plot2d(nx, xn, style=-5); gce().children.mark_foreground = 2;
    xgrid(); xlabel('n'); ylabel('x(n)'); title('x(n) - Tin hieu goc');
    // Plot y(n) - delayed signal
    subplot(2,1,2);
    plot2d3(ny, yn, style=5); gce().children.thickness = 2;
    plot2d(ny, yn, style=-5); gce().children.mark_foreground = 5;
    xgrid(); xlabel('n'); ylabel('y(n)');
    title('y(n) = x(n - ' + string(k) + ')');
endfunction

// ------------------------------------------------------------
// x(n) spans n = -3,-2,-1, 0, 1, 2, 3, 4, 5, 6
// ------------------------------------------------------------
[yn, yorigin] = delay(xn, xorigin, 2);
disp('yn =');     disp(yn);
disp('yorigin ='); disp(yorigin);
```



Hình 11: *Tín hiệu gốc $x(n)$ và tín hiệu trễ $x(n-2)$*

(b) $x(4-n)$ - Fold rồi Delay 4 mẫu

Phân tích: $x(4-n) = x(-(n-4))$, thực hiện lần lượt hai bước:

- **Bước 1 - Fold:** Thay $n \to -n$, lật ngược tín hiệu quanh trục $n = 0$:
- **Bước 2 - Delay 4 mẫu:** Thay $n \to n-4$, dịch phải 4 bước:

$$x(n) \xrightarrow{n \to -n} x(-n) \xrightarrow{n \to n-4} x(-(n-4)) = x(4-n)$$

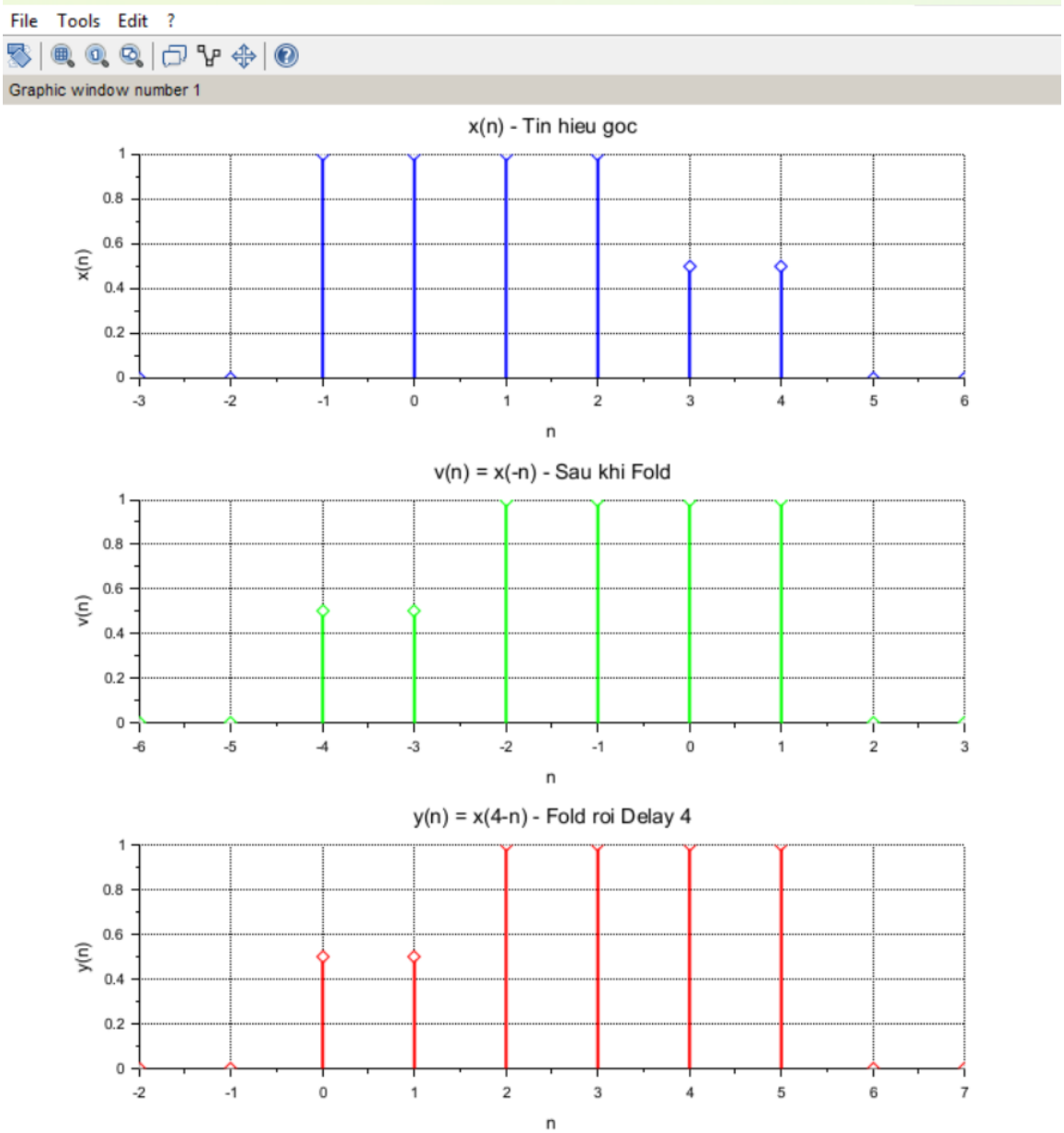
Bảng giá trị:

n	$\cdots$	-1	0	1	2	3	4	5	$\cdots$
$m = 4-n$		5	4	3	2	1	0	-1	
$x(4-n)$	0	0	$\frac{1}{2}$	$\frac{1}{2}$	1	1	1	1	0

Bảng 7: *Giá trị tín hiệu $x(4-n)$*

```
// ============================================================
// Define signal x(n) from Figure P2.2
// ============================================================
xn      = [0, 0, 1, 1, 1, 1, 0.5, 0.5, 0, 0];
xorigin = 4;                    // n=0 is at index 4

// ============================================================
// FUNCTION : fold
// ============================================================
function [yn, yorigin] = fold(xn, xorigin)
    Nx      = length(xn);
    yn      = xn($:-1:1);           // reverse the sample vector
    yorigin = Nx - xorigin + 1;     // mirror the origin position
endfunction

// ============================================================
// FUNCTION : delay
// ============================================================
function [yn, yorigin] = delay(xn, xorigin, k)
    yn      = xn;                   // sample values unchanged
    yorigin = xorigin - k;          // origin index shifts left by k
endfunction

// ------------------------------------------------------------
// Step 1: Fold x(n) -> v(n) = x(-n)
// Step 2: Delay v(n) by 4 -> y(n) = v(n-4) = x(4-n)
// ------------------------------------------------------------
[vn, vorigin] = fold(xn, xorigin);
[yn_b, yorigin_b] = delay(vn, vorigin, 4);

// Build time axes for plotting
Nx = length(xn);
nx   = (1:Nx) - xorigin;           // time axis of x(n)
nv   = (1:Nx) - vorigin;           // time axis of v(n) = x(-n)
ny_b = (1:Nx) - yorigin_b;         // time axis of y(n) = x(4-n)

// Plot all 3 signals in one figure
scf(); clf();
subplot(3,1,1);
plot2d3(nx, xn, style=2); gce().children.thickness = 2;
plot2d(nx, xn, style=-5); gce().children.mark_foreground = 2;
xgrid(); xlabel('n'); ylabel('x(n)'); title('x(n) - Tin hieu goc');

subplot(3,1,2);
plot2d3(nv, vn, style=3); gce().children.thickness = 2;
```

```
plot2d(nv, vn, style=-5); gce().children.mark_foreground = 3;
xgrid(); xlabel('n'); ylabel('v(n)');
title('v(n) = x(-n) - Sau khi Fold');

subplot(3,1,3);
plot2d3(ny_b, yn_b, style=5); gce().children.thickness = 2;
plot2d(ny_b, yn_b, style=-5); gce().children.mark_foreground = 5;
xgrid(); xlabel('n'); ylabel('y(n)');
title('y(n) = x(4-n) - Fold roi Delay 4');

disp('yn_b = ');     disp(yn_b);
disp('yorigin = '); disp(yorigin_b);
```



Hình 12: *Tín hiệu gốc $x(n)$ và tín hiệu $x(4-n)$ thu được sau khi Fold rồi Delay 4 mẫu.*

(c) $x(n+2)$ - Advance 2 mẫu

Phân tích: Thay $n \to n+2$, dịch chuyển tín hiệu sang **trái 2 bước** trên trục n:

$$x(n) \xrightarrow{n \to n+2} x(n+2) \quad \text{xác định trên } [-3,\ 2]$$

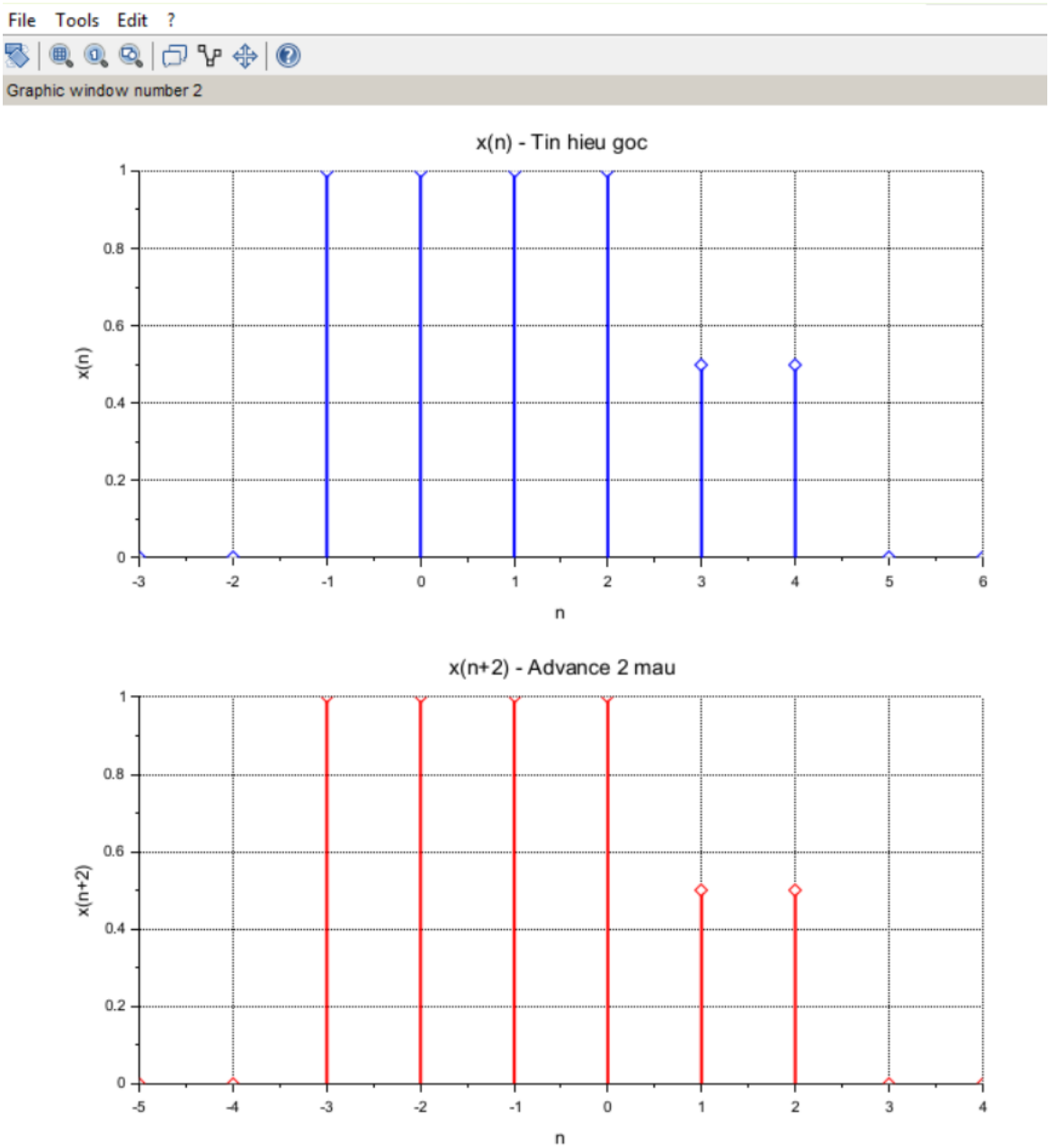
Bảng giá trị:

n	$\cdots$	-4	-3	-2	-1	0	1	2	$\cdots$
$m = n+2$		-2	-1	0	1	2	3	4	
$x(n+2)$	0	0	1	1	1	1	$\frac{1}{2}$	$\frac{1}{2}$	0

Bảng 8: *Giá trị tín hiệu $x(n+2)$*

```
// ============================================================
// Define signal x(n) from Figure P2.2
// ============================================================
xn      = [0, 0, 1, 1, 1, 1, 0.5, 0.5, 0, 0];
xorigin = 4;                      // n=0 is at index 4

// ============================================================
// FUNCTION : advance
// ============================================================
function [yn, yorigin] = advance(xn, xorigin, k)
    yn      = xn;                     // sample values unchanged
    yorigin = xorigin + k;            // origin index shifts right by k
endfunction

// ------------------------------------------------------------
// x(n) spans n = -3,-2,-1, 0, 1, 2, 3, 4, 5, 6
// Advance k=2: y(n) spans same vector, yorigin = 4+2 = 6
// y(n) xac dinh tai n = -3,-2,-1, 0, 1, 2 (shifted left by 2)
// ------------------------------------------------------------
[yn_c, yorigin_c] = advance(xn, xorigin, 2);

// Build time axes for plotting
nx   = (1:length(xn)) - xorigin;      // time axis of x(n)
ny_c = (1:length(yn_c)) - yorigin_c; // time axis of y(n)

// Plot both signals in one figure
scf(); clf();
subplot(2,1,1);
plot2d3(nx, xn, style=2); gce().children.thickness = 2;
plot2d(nx, xn, style=-5); gce().children.mark_foreground = 2;
xgrid(); xlabel('n'); ylabel('x(n)'); title('x(n) - Tin hieu goc');
```

```
subplot(2,1,2);
plot2d3(ny_c, yn_c, style=5); gce().children.thickness = 2;
plot2d(ny_c, yn_c, style=-5); gce().children.mark_foreground = 5;
xgrid(); xlabel('n'); ylabel('x(n+2)'); title('x(n+2) - Advance 2 mau');

disp('yn_c =');     disp(yn_c);
disp('yorigin ='); disp(yorigin_c);
```



Hình 13: *Tín hiệu gốc $x(n)$ và tín hiệu $x(n+2)$*

(d) $x(n) \cdot u(2-n)$ - Nhân với hàm bước đơn vị

Phân tích: $u(2-n)$ là hàm bước đơn vị dịch chuyển, xác định:

$$u(2-n) = \begin{cases} 1, & n \leq 2 \\ 0, & n > 2 \end{cases}$$

Nhân $x(n)$ với $u(2-n)$ có tác dụng **cắt bỏ** toàn bộ phần tín hiệu tại $n > 2$:

$$y(n) = x(n) \cdot u(2-n) = \begin{cases} x(n), & n \leq 2 \\ 0, & n > 2 \end{cases}$$

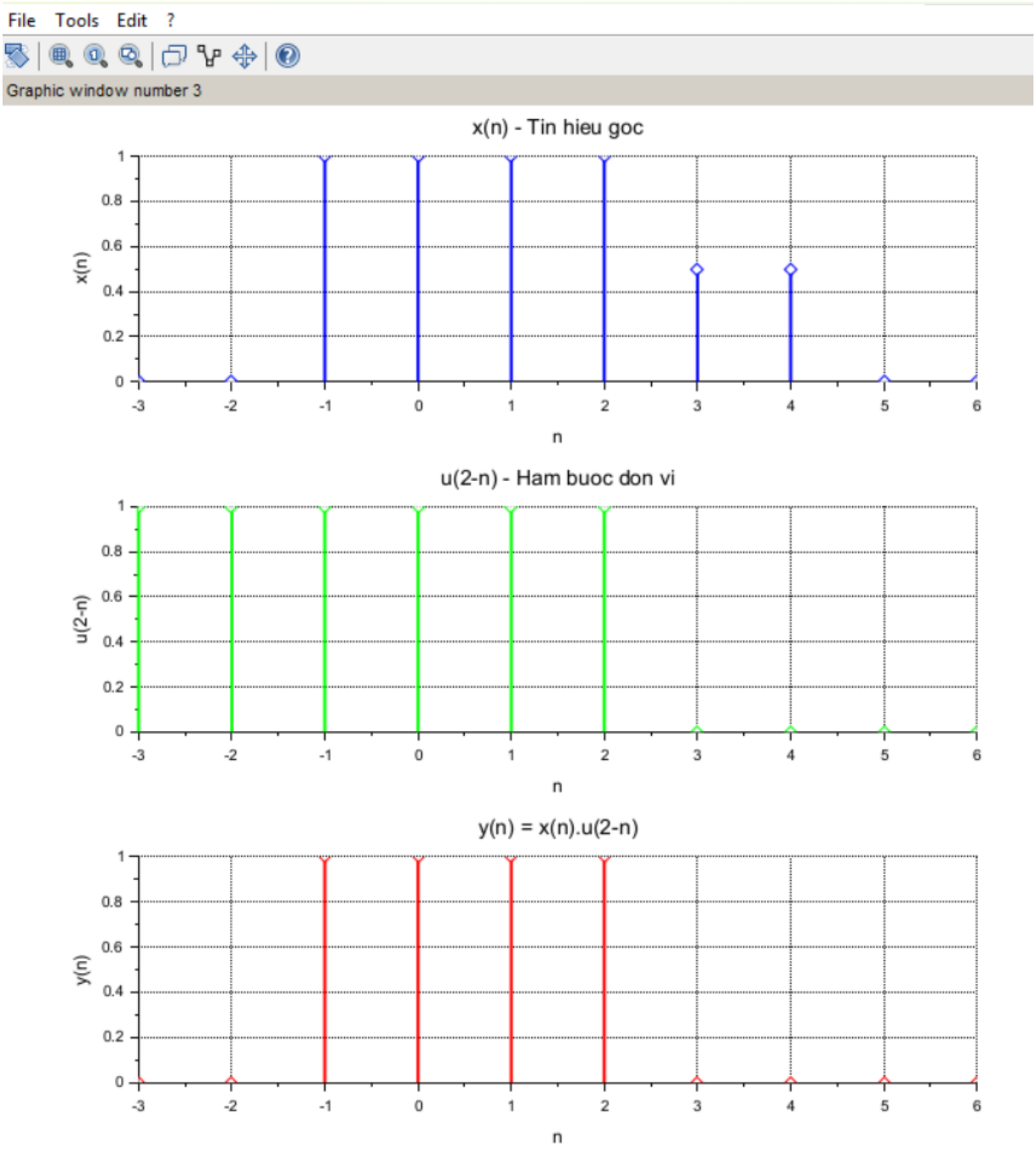
Bảng giá trị:

n	$\cdots$	-1	0	1	2	3	4	$\cdots$
$x(n)$	0	1	1	1	1	$\frac{1}{2}$	$\frac{1}{2}$	0
$u(2-n)$	1	1	1	1	1	0	0	0
$x(n) \cdot u(2-n)$	0	1	1	1	1	0	0	0

Bảng 9: *Giá trị tín hiệu* $x(n) \cdot u(2-n)$

```
// ============================================================
// Define signal x(n) from Figure P2.2
// ============================================================
xn      = [0, 0, 1, 1, 1, 1, 0.5, 0.5, 0, 0];
xorigin = 4;                        // n=0 is at index 4

// ============================================================
// FUNCTION : multi
// ============================================================
function [yn, yorigin] = multi(x1n, x1origin, x2n, x2origin)
    n1 = (1:length(x1n)) - x1origin;
    n2 = (1:length(x2n)) - x2origin;
    nmin = min(min(n1), min(n2));
    nmax = max(max(n1), max(n2));
    ny   = nmin:nmax;
    yorigin = 1 - nmin;
    y1 = zeros(1, length(ny));
    y2 = zeros(1, length(ny));
    y1(n1 - nmin + 1) = x1n;            // zero-pad x1(n)
    y2(n2 - nmin + 1) = x2n;            // zero-pad x2(n)
    yn = y1 .* y2;                       // element-wise multiplication
endfunction

```

```
// ------------------------------------------------------------
// Build u(2-n): equals 1 for n <= 2, 0 for n > 2
// Define on same range as xn: n = -3..6
// ------------------------------------------------------------
n_axis = (1:length(xn)) - xorigin;      // time axis: -3..6
u2n    = double(n_axis <= 2);           // u(2-n) = 1 when n <= 2

// Multiply x(n) . u(2-n)
[yn_d, yorigin_d] = multi(xn, xorigin, u2n, xorigin);

// Build time axes for plotting
nx   = (1:length(xn))   - xorigin;
ny_d = (1:length(yn_d)) - yorigin_d;

// Plot all signals in one figure
scf(); clf();
subplot(3,1,1);
plot2d3(nx, xn, style=2); gce().children.thickness = 2;
plot2d(nx, xn, style=-5); gce().children.mark_foreground = 2;
xgrid(); xlabel('n'); ylabel('x(n)');
title('x(n) - Tin hieu goc');
subplot(3,1,2);

plot2d3(nx, u2n, style=3); gce().children.thickness = 2;
plot2d(nx, u2n, style=-5); gce().children.mark_foreground = 3;
xgrid(); xlabel('n'); ylabel('u(2-n)');
title('u(2-n) - Ham buoc don vi');
subplot(3,1,3);

plot2d3(ny_d, yn_d, style=5); gce().children.thickness = 2;
plot2d(ny_d, yn_d, style=-5); gce().children.mark_foreground = 5;
xgrid(); xlabel('n'); ylabel('y(n)');
title('y(n) = x(n).u(2-n)');
```



Hình 14: *Tín hiệu $x(n)$, hàm bước $u(2-n)$ và tín hiệu tích $y(n) = x(n) \cdot u(2-n)$*

(e) $x(n-1) \cdot \delta(n-3)$ - Nhân với xung dịch chuyển

Phân tích: $\delta(n-3)$ chỉ bằng 1 tại $n = 3$ và bằng 0 ở mọi nơi khác:

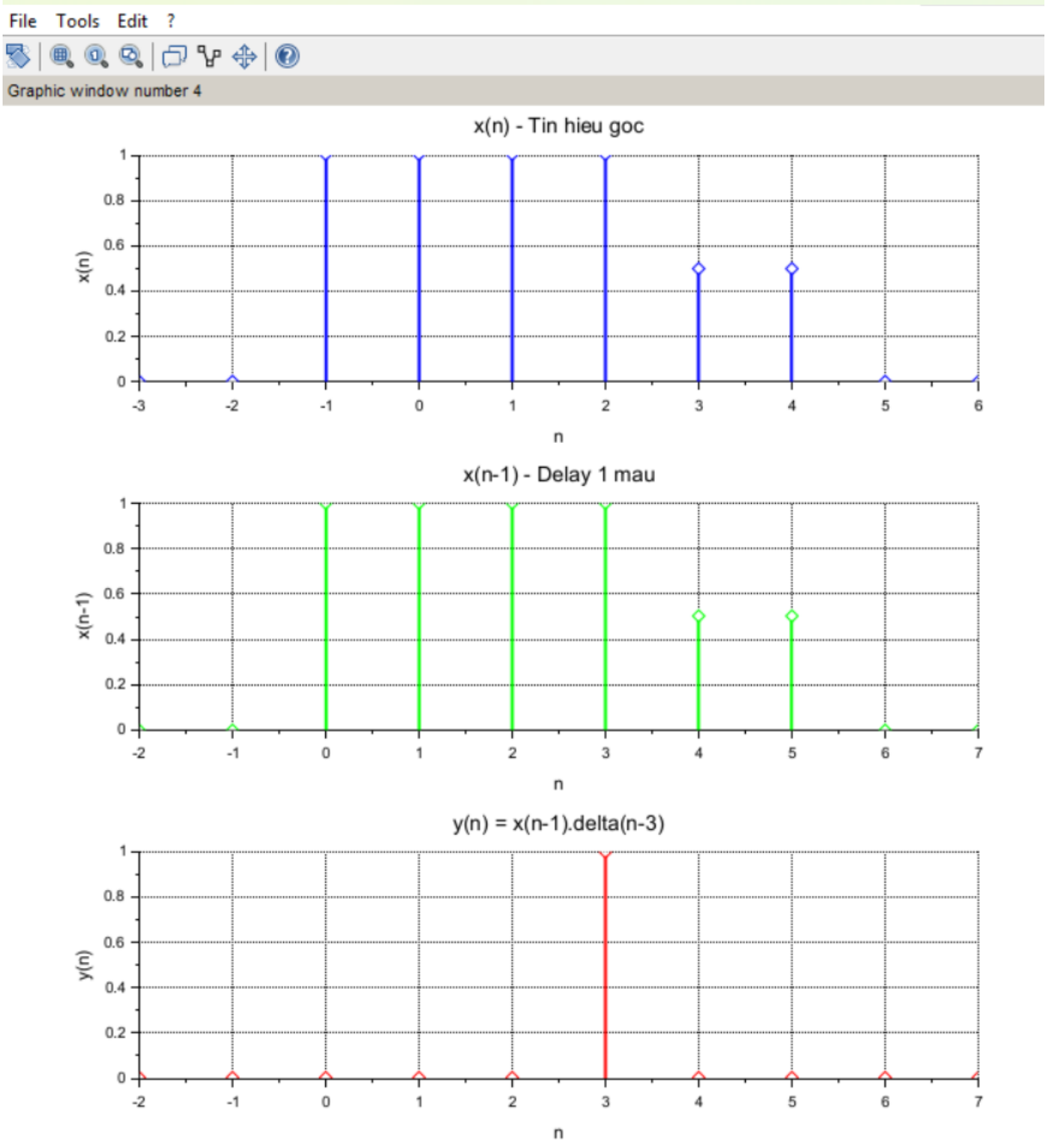
$$\delta(n-3) = \begin{cases} 1, & n = 3 \\ 0, & n \neq 3 \end{cases}$$

Bảng giá trị:

n	$\cdots$	0	1	2	3	4	5	$\cdots$
$x(n-1)$	0	1	1	1	1	$\frac{1}{2}$	$\frac{1}{2}$	0
$\delta(n-3)$	0	0	0	0	1	0	0	0
$x(n-1)\cdot\delta(n-3)$	0	0	0	0	1	0	0	0

Bảng 10: *Giá trị tín hiệu* $x(n-1)\cdot\delta(n-3)$

```
// ============================================================
// (e): x(n-1) * delta(n-3)
// ============================================================
// Step 1: Delay x(n) by 1 -> v(n) = x(n-1)
[vn_e, vorigin_e] = delay(xn, xorigin, 1);

// Step 2: Build delta(n-3) on same axis as v(n)
nv_axis  = (1:length(vn_e)) - vorigin_e;   // time axis of v(n)
delta_n3 = double(nv_axis == 3);           // delta(n-3) = 1 only at n=3

// Step 3: Multiply v(n) . delta(n-3)
[yn_e, yorigin_e] = multi(vn_e, vorigin_e, delta_n3, vorigin_e);

// Build time axes for plotting
nx    = (1:length(xn))   - xorigin;
nv_e  = (1:length(vn_e)) - vorigin_e;
ny_e  = (1:length(yn_e)) - yorigin_e;

// Plot all signals in one figure
scf(); clf();
subplot(3,1,1);
plot2d3(nx, xn, style=2); gce().children.thickness = 2;
plot2d(nx, xn, style=-5); gce().children.mark_foreground = 2;
xgrid(); xlabel('n'); ylabel('x(n)'); title('x(n) - Tin hieu goc');

subplot(3,1,2);
plot2d3(nv_e, vn_e, style=3); gce().children.thickness = 2;
plot2d(nv_e, vn_e, style=-5); gce().children.mark_foreground = 3;
xgrid(); xlabel('n'); ylabel('x(n-1)'); title('x(n-1) - Delay 1 mau');

subplot(3,1,3);
plot2d3(ny_e, yn_e, style=5); gce().children.thickness = 2;
plot2d(ny_e, yn_e, style=-5); gce().children.mark_foreground = 5;
xgrid(); xlabel('n'); ylabel('y(n)'); title('y(n) = x(n-1).delta(n-3)');
```



Hình 15: *Tín hiệu $x(n-1)$ và tích $y(n) = x(n-1) \cdot \delta(n-3)$*

(f) $x(n^2)$ - Tái lấy mẫu phi tuyến

Phân tích: Thay $n \to n^2$, tín hiệu được lấy mẫu lại tại các vị trí n^2 trên trục thời gian. Vì $x(n) \neq 0$ chỉ trên $[-1,\ 4]$, cần tìm các giá trị n sao cho $n^2 \in [-1,\ 4]$:

$$n^2 \in [0,\ 4] \implies n \in \{-2,\ -1,\ 0,\ 1,\ 2\}$$

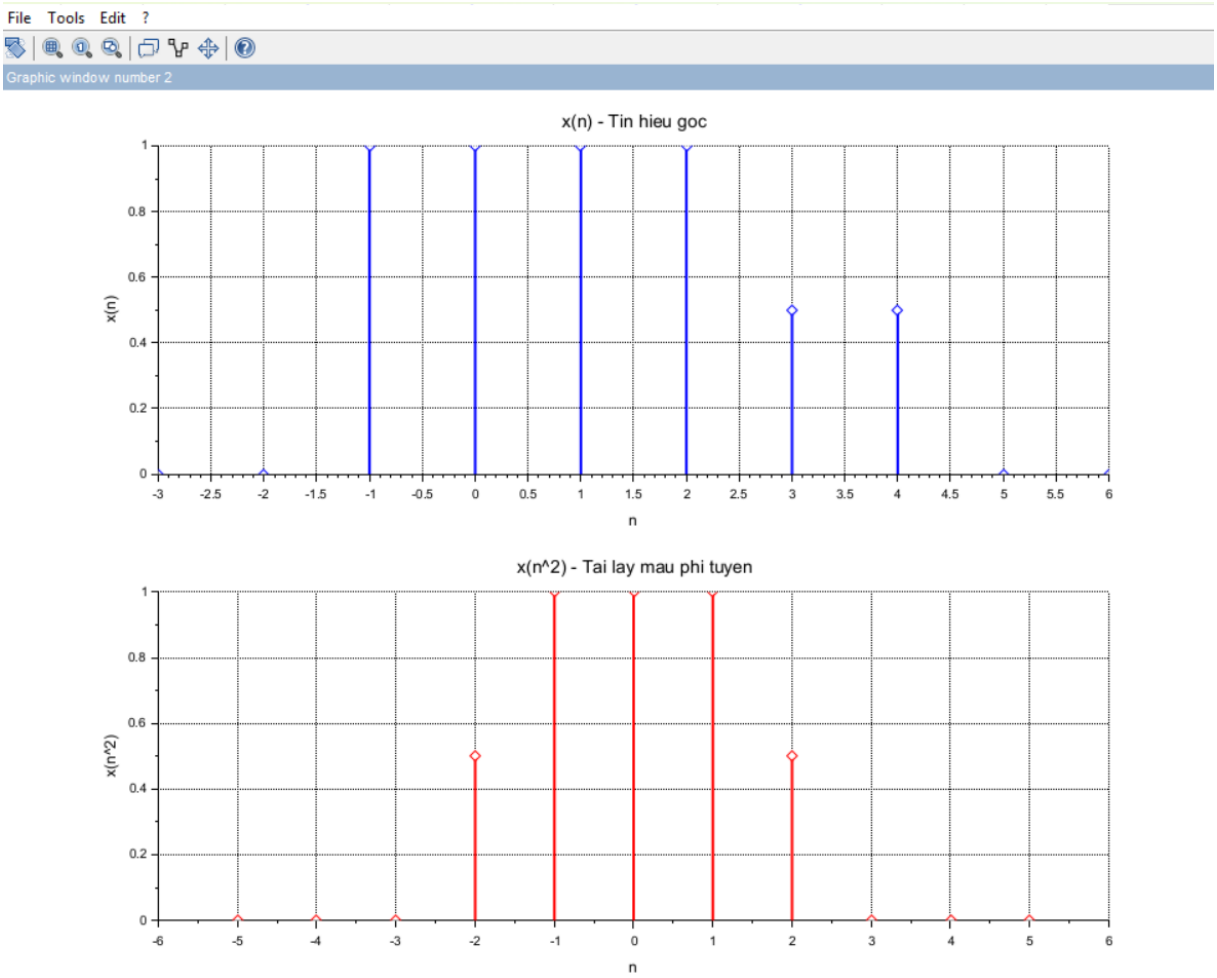
Bảng giá trị:

n	-2	-1	0	1	2
n^2	4	1	0	1	4
$x(n^2)$	$\frac{1}{2}$	1	1	1	$\frac{1}{2}$

Bảng 11: *Giá trị tín hiệu $x(n^2)$*

Nhận xét: Tín hiệu $x(n^2)$ có tính **đối xứng chẵn** vì $n^2 = (-n)^2$, do đó $x(n^2) = x((-n)^2)$ với mọi n.

```
// Define signal x(n) from Figure P2.2
xn       = [0, 0, 1, 1, 1, 1, 0.5, 0.5, 0, 0];
xorigin = 4;                        // n=0 is at index 4

n_f     = -5:5;
yn_f    = zeros(1, length(n_f));
n_axis = (1:length(xn)) - xorigin;      // original time axis of x(n)

for i = 1:length(n_f)
    m    = n_f(i)^2;                    // compute n^2
    idx = find(n_axis == m);            // find position of n^2 in x(n)
    if length(idx) > 0 then             // Scilab: use length() instead of ~isempty
        yn_f(i) = xn(idx);              // assign x(n^2)
    else
        yn_f(i) = 0;                    // zero if n^2 outside support
    end
end

// Build time axes for plotting
nx = (1:length(xn)) - xorigin;          // time axis of x(n)
// Plot both signals in one figure
scf(); clf();
subplot(2,1,1);
plot2d3(nx, xn, style=2); gce().children.thickness = 2;
plot2d(nx, xn, style=-5); gce().children.mark_foreground = 2;
xgrid(); xlabel('n'); ylabel('x(n)'); title('x(n) - Tin hieu goc');

subplot(2,1,2);
plot2d3(n_f, yn_f, style=5); gce().children.thickness = 2;
plot2d(n_f, yn_f, style=-5); gce().children.mark_foreground = 5;
xgrid(); xlabel('n'); ylabel('x(n^2)'); title('x(n^2) - Tai lay mau phi tuyen');
```



Hình 16: *Tín hiệu gốc $x(n)$ và tín hiệu $x(n^2)$ sau khi tái lấy mẫu phi tuyến*

(g) Thành phần chẵn của $x(n)$

Thành phần chẵn của tín hiệu $x(n)$ được định nghĩa:

$$x_e(n) = \frac{x(n) + x(-n)}{2}$$

Tính $x(-n)$ bằng cách fold tín hiệu $x(n)$, sau đó cộng với $x(n)$ rồi chia đôi.

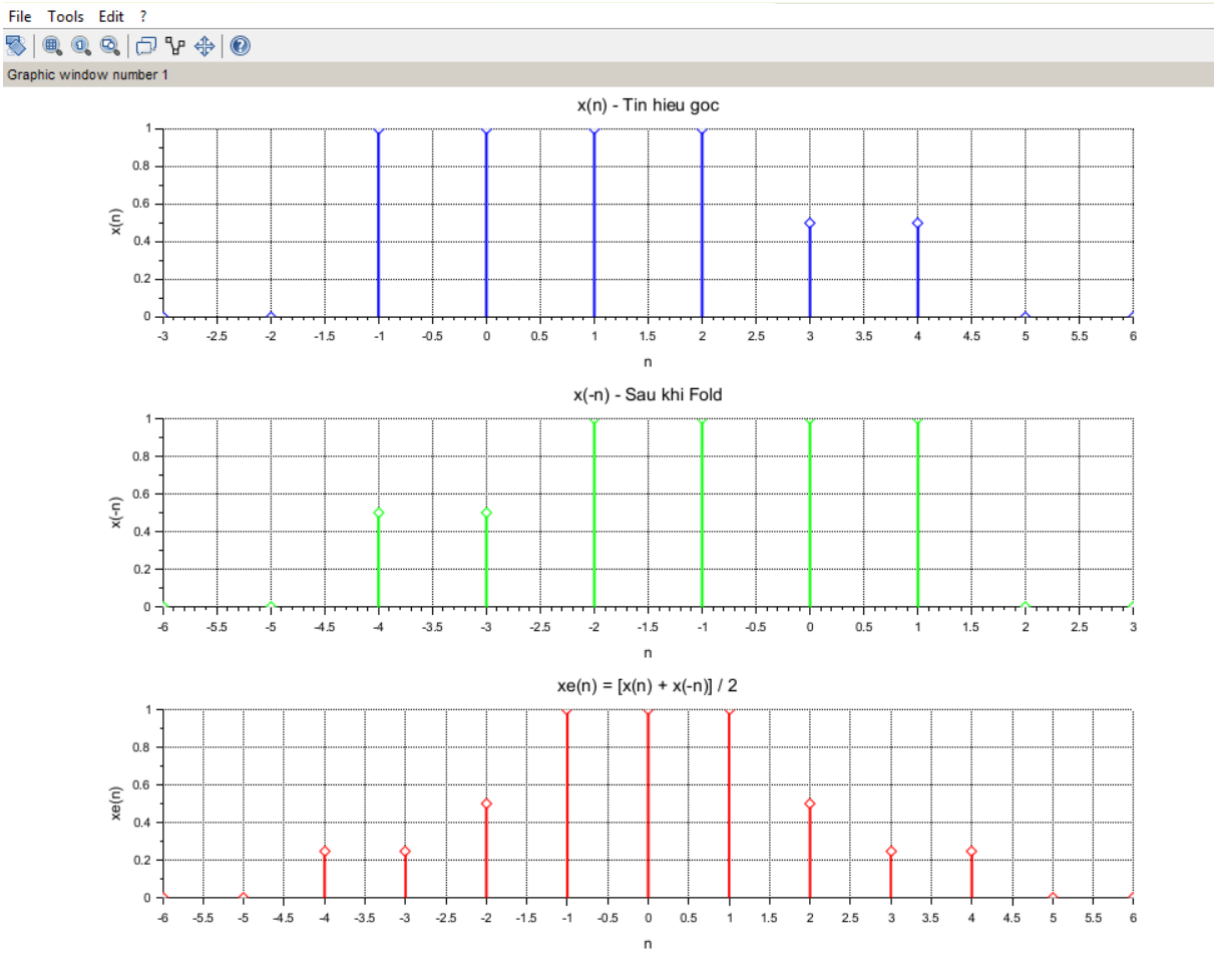
Bảng giá trị:

n	-4	-3	-2	-1	0	1	2	3	4
$x(n)$	0	0	0	1	1	1	1	$\frac{1}{2}$	$\frac{1}{2}$
$x(-n)$	$\frac{1}{2}$	$\frac{1}{2}$	1	1	1	1	0	0	0
$x_e(n)$	$\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{2}$	1	1	1	$\frac{1}{2}$	$\frac{1}{4}$	$\frac{1}{4}$

Bảng 12: *Giá trị thành phần chẵn $x_e(n)$*

```
// ============================================================
// Define signal x(n) from Figure P2.2
// ============================================================
xn      = [0, 0, 1, 1, 1, 1, 0.5, 0.5, 0, 0];
xorigin = 4;

// ============================================================
// FUNCTION: fold - chi tinh toan, khong ve
// ============================================================
function [yn, yorigin] = fold(xn, xorigin)
    Nx      = length(xn);
    yn      = xn($:-1:1);           // reverse the sample vector
    yorigin = Nx - xorigin + 1;     // mirror the origin position
endfunction

// ============================================================
// FUNCTION: add - chi tinh toan, khong ve
// ============================================================
function [yn, yorigin] = add(x1n, x1origin, x2n, x2origin)
    n1 = (1:length(x1n)) - x1origin;
    n2 = (1:length(x2n)) - x2origin;
    nmin = min(min(n1), min(n2));
    nmax = max(max(n1), max(n2));
    ny   = nmin:nmax;
    yorigin = 1 - nmin;
    y1 = zeros(1, length(ny));
    y2 = zeros(1, length(ny));
    y1(n1 - nmin + 1) = x1n;
    y2(n2 - nmin + 1) = x2n;
    yn = y1 + y2;
endfunction

// ============================================================
// (g): Even part xe(n) = [x(n) + x(-n)] / 2
// ============================================================

// Step 1: Fold x(n) -> xfold(n) = x(-n)
[xfold, xfold_origin] = fold(xn, xorigin);

// Step 2: Add x(n) + x(-n) on union time axis
[xsum, xsum_origin] = add(xn, xorigin, xfold, xfold_origin);

// Step 3: Divide by 2 -> even part
xe        = xsum / 2;
xe_origin = xsum_origin;
```

```

// Build time axes for plotting
nx = (1:length(xn))     - xorigin;
nf = (1:length(xfold)) - xfold_origin;
ne = (1:length(xe))     - xe_origin;

// Plot all signals in one figure
scf(); clf();
subplot(3,1,1);
plot2d3(nx, xn, style=2); gce().children.thickness = 2;
plot2d(nx, xn, style=-5); gce().children.mark_foreground = 2;
xgrid(); xlabel('n'); ylabel('x(n)'); title('x(n) - Tin hieu goc');

subplot(3,1,2);
plot2d3(nf, xfold, style=3); gce().children.thickness = 2;
plot2d(nf, xfold, style=-5); gce().children.mark_foreground = 3;
xgrid(); xlabel('n'); ylabel('x(-n)'); title('x(-n) - Sau khi Fold');

subplot(3,1,3);
plot2d3(ne, xe, style=5); gce().children.thickness = 2;
plot2d(ne, xe, style=-5); gce().children.mark_foreground = 5;
xgrid(); xlabel('n'); ylabel('xe(n)'); title('xe(n) = [x(n) + x(-n)] / 2
    ');

disp('xe =');          disp(xe);
disp('xe_origin =');  disp(xe_origin);
```



Hình 17: *Tín hiệu gốc $x(n)$, tín hiệu $x(-n)$ và thành phần chẵn $x_e(n)$*

(h) Thành phần lẻ của $x(n)$

Thành phần lẻ của tín hiệu $x(n)$ được định nghĩa:

$$x_o(n) = \frac{x(n) - x(-n)}{2}$$

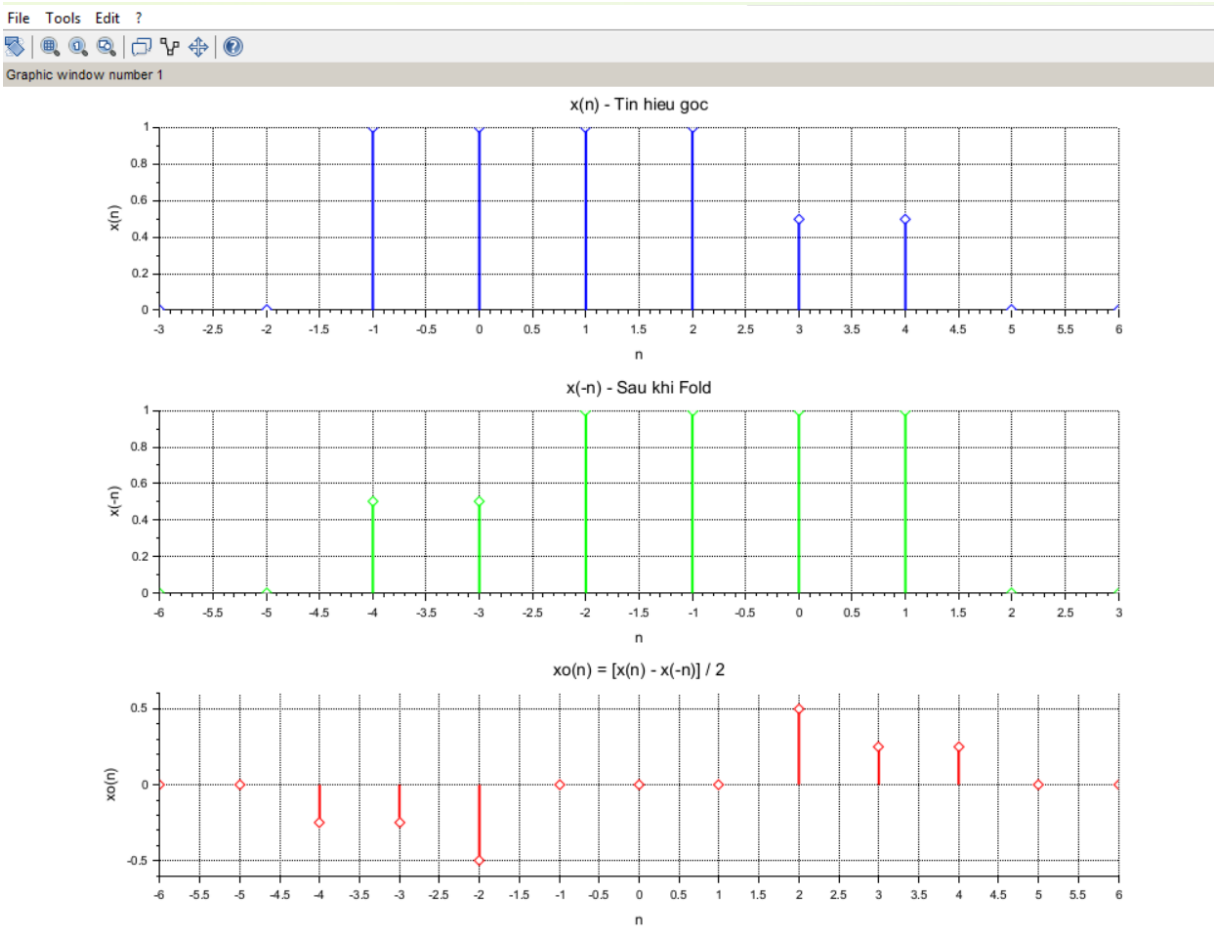
Bảng giá trị:

n	-4	-3	-2	-1	0	1	2	3	4
$x(n)$	0	0	0	1	1	1	1	$\frac{1}{2}$	$\frac{1}{2}$
$x(-n)$	$\frac{1}{2}$	$\frac{1}{2}$	1	1	1	1	0	0	0
$x_o(n)$	$-\frac{1}{4}$	$-\frac{1}{4}$	$-\frac{1}{2}$	0	0	0	$\frac{1}{2}$	$\frac{1}{4}$	$\frac{1}{4}$

Bảng 13: *Giá trị thành phần lẻ $x_o(n)$*

```
// ============================================================
// Define signal x(n) from Figure P2.2
// ============================================================
xn      = [0, 0, 1, 1, 1, 1, 0.5, 0.5, 0, 0];
xorigin = 4;                    // n=0 is at index 4

// ============================================================
// FUNCTION: fold - chi tinh toan, khong ve
// ============================================================
function [yn, yorigin] = fold(xn, xorigin)
    Nx      = length(xn);
    yn      = xn($:-1:1);           // reverse the sample vector
    yorigin = Nx - xorigin + 1;     // mirror the origin position
endfunction

// ============================================================
// FUNCTION: add - chi tinh toan, khong ve
// ============================================================
function [yn, yorigin] = add(x1n, x1origin, x2n, x2origin)
    n1   = (1:length(x1n)) - x1origin;
    n2   = (1:length(x2n)) - x2origin;
    nmin = min(min(n1), min(n2));
    nmax = max(max(n1), max(n2));
    ny   = nmin:nmax;
    yorigin = 1 - nmin;
    y1 = zeros(1, length(ny));
    y2 = zeros(1, length(ny));
    y1(n1 - nmin + 1) = x1n;        // zero-pad x1(n)
    y2(n2 - nmin + 1) = x2n;        // zero-pad x2(n)
    yn = y1 + y2;
endfunction

// ============================================================
// (h): Odd part xo(n) = [x(n) - x(-n)] / 2
// Step 1: Fold x(n) -> x(-n)
// Step 2: Subtract x(n) - x(-n) using add() with negated xfold
// Step 3: Divide by 2
// ============================================================

// Step 1: Fold x(n) -> xfold(n) = x(-n)
[xfold, xfold_origin] = fold(xn, xorigin);

// Step 2: x(n) - x(-n) = x(n) + (-x(-n))
[xdiff, xdiff_origin] = add(xn, xorigin, -xfold, xfold_origin);

```

```
// Step 3: Divide by 2 -> odd part
xo        = xdiff / 2;
xo_origin = xdiff_origin;

// Build time axes for plotting
nx = (1:length(xn))    - xorigin;
nf = (1:length(xfold)) - xfold_origin;
no = (1:length(xo))    - xo_origin;

// Plot all signals in one figure
scf(); clf();
subplot(3,1,1);
plot2d3(nx, xn, style=2); gce().children.thickness = 2;
plot2d(nx, xn, style=-5); gce().children.mark_foreground = 2;
xgrid(); xlabel('n'); ylabel('x(n)'); title('x(n) - Tin hieu goc');

subplot(3,1,2);
plot2d3(nf, xfold, style=3); gce().children.thickness = 2;
plot2d(nf, xfold, style=-5); gce().children.mark_foreground = 3;
xgrid(); xlabel('n'); ylabel('x(-n)'); title('x(-n) - Sau khi Fold');

subplot(3,1,3);
plot2d3(no, xo, style=5); gce().children.thickness = 2;
plot2d(no, xo, style=-5); gce().children.mark_foreground = 5;
xgrid(); xlabel('n'); ylabel('xo(n)'); title('xo(n) = [x(n) - x(-n)] / 2
    ');

disp('xo =');         disp(xo);
disp('xo_origin ='); disp(xo_origin);

// ------------------------------------------------------------
// Verify: xe(n) + xo(n) = x(n)
// ------------------------------------------------------------
[xsum, xsum_origin]   = add(xn, xorigin, xfold, xfold_origin);
xe                    = xsum / 2;
[xcheck, xcheck_orig] = add(xe, xsum_origin, xo, xo_origin);
disp('Kiem tra xe(n) + xo(n) (phai bang x(n)):');
disp(xcheck);
```



Hình 18: *Tín hiệu gốc $x(n)$, tín hiệu $x(-n)$ và thành phần lẻ $x_o(n)$*

5.3 Problem 2.4: Phân rã tín hiệu thành phần chẵn và lẻ

Yêu cầu đề bài

- **Chứng minh rằng mọi tín hiệu rời rạc $x(n)$ đều có thể được phân rã thành hai thành phần:**
  - **thành phần chẵn $x_e(n)$**
  - **thành phần lẻ $x_o(n)$**

  Trong đó:

$$x(n) = x_e(n) + x_o(n)$$

- **Thảo luận xem phép phân rã này có duy nhất hay không.**
- **Áp dụng cho tín hiệu:**

$$x(n) = \{2, 3, 4 \uparrow, 5, 6\}$$

  Xác định và biểu diễn đồ thị của:
  - **tín hiệu chẵn $x_e(n)$**
  - **tín hiệu lẻ $x_o(n)$**

Chứng minh: Mọi tín hiệu rời rạc $x(n)$ đều phân rã

Cho tín hiệu rời rạc $x(n)$ bất kỳ. Ta định nghĩa hai tín hiệu:

$$\boxed{x_e(n) = \frac{x(n) + x(-n)}{2}} \qquad \boxed{x_o(n) = \frac{x(n) - x(-n)}{2}} \tag{1}$$

Bước 1 - Chứng minh $x_e(n)$ là tín hiệu chẵn:

$$x_e(-n) = \frac{x(-n) + x\big(-(-n)\big)}{2} = \frac{x(-n) + x(n)}{2} = \frac{x(n) + x(-n)}{2} = x_e(n) \quad \checkmark \tag{2}$$

Bước 2 - Chứng minh $x_o(n)$ là tín hiệu lẻ:

$$x_o(-n) = \frac{x(-n) - x\big(-(-n)\big)}{2} = \frac{x(-n) - x(n)}{2} = -\frac{x(n) - x(-n)}{2} = -x_o(n) \quad \checkmark \tag{3}$$

Bước 3 - Chứng minh tái tạo hoàn toàn:

$$x_e(n) + x_o(n) = \frac{x(n) + x(-n)}{2} + \frac{x(n) - x(-n)}{2} = \frac{x(n) + x(-n) + x(n) - x(-n)}{2} = \frac{2x(n)}{2} = x(n) \quad \checkmark \tag{4}$$

Kết luận: Với mọi tín hiệu rời rạc $x(n)$, luôn tồn tại cách phân rã thành tổng của thành phần chẵn $x_e(n)$ và thành phần lẻ $x_o(n)$.

Chứng minh: Phép phân rã là duy nhất

Giả sử phản chứng: Phép phân rã không duy nhất, tồn tại hai cách phân rã khác nhau của cùng $x(n)$:

$$x(n) = x_{e1}(n) + x_{o1}(n) \qquad \big(x_{e1} \text{ chẵn},\ x_{o1} \text{ lẻ}\big) \tag{1}$$

$$x(n) = x_{e2}(n) + x_{o2}(n) \qquad \big(x_{e2} \text{ chẵn},\ x_{o2} \text{ lẻ}\big) \tag{2}$$

Lấy (1) − (2):

$$x_{e1}(n) - x_{e2}(n) = x_{o2}(n) - x_{o1}(n) \tag{3}$$

Đặt:

$$g(n) = x_{e1}(n) - x_{e2}(n) \quad \Rightarrow \quad g(n) \text{ là hiệu hai hàm chẵn} \Rightarrow g(n) \textbf{ chẵn} \tag{4}$$

$$h(n) = x_{o2}(n) - x_{o1}(n) \quad \Rightarrow \quad h(n) \text{ là hiệu hai hàm lẻ} \Rightarrow h(n) \textbf{ lẻ} \tag{5}$$

Ta có $g(n) = h(n)$ với mọi n. Xét tại $-n$:

$$g(-n) = h(-n) \tag{6}$$

Áp dụng tính chẵn của g và tính lẻ của h:

$$g(n) = -h(n) \tag{7}$$

Kết hợp với $g(n) = h(n)$:

$$h(n) = -h(n) \implies 2h(n) = 0 \implies h(n) = 0 \implies g(n) = 0 \quad \forall n \tag{8}$$

Từ (4), (5) và (8) suy ra:

$$x_{e1}(n) = x_{e2}(n) \qquad \text{và} \qquad x_{o1}(n) = x_{o2}(n) \tag{9}$$

Điều này mâu thuẫn với giả sử ban đầu.

Kết luận: Phép phân rã tín hiệu thành thành phần chẵn và thành phần lẻ là **duy nhất**.

Áp dụng cho $x(n) = \{2,\ 3,\ \overset{\uparrow}{4},\ 5,\ 6\}$

Bước 1 - Xác định tín hiệu gốc

Ký hiệu $\uparrow$ tại giá trị 4 xác định **gốc thời gian** $n = 0$:

n	-2	-1	0	1	2
$x(n)$	2	3	**4** ($\uparrow$)	5	6

Bảng 14: *Tín hiệu gốc* $x(n) = \{2,\ 3,\ \overset{\uparrow}{4},\ 5,\ 6\}$

Bước 2 - Tính $x(-n)$

Lật tín hiệu quanh $n = 0$:

n	-2	-1	0	1	2
$x(n)$	2	3	**4** (↑)	5	6
$x(-n)$	6	5	**4** (↑)	3	2

Bảng 15: *So sánh $x(n)$ và $x(-n)$ — tín hiệu gốc và tín hiệu lật*

Bước 3 - Tính $x_e(n)$ và $x_o(n)$

n	$x(n)$	$x(-n)$	$x_e(n) = \dfrac{x(n)+x(-n)}{2}$	$x_o(n) = \dfrac{x(n)-x(-n)}{2}$
-2	2	6	$\dfrac{2+6}{2} = \mathbf{4}$	$\dfrac{2-6}{2} = \mathbf{-2}$
-1	3	5	$\dfrac{3+5}{2} = \mathbf{4}$	$\dfrac{3-5}{2} = \mathbf{-1}$
0	4	4	$\dfrac{4+4}{2} = \mathbf{4}$	$\dfrac{4-4}{2} = \mathbf{0}$
1	5	3	$\dfrac{5+3}{2} = \mathbf{4}$	$\dfrac{5-3}{2} = \mathbf{1}$
2	6	2	$\dfrac{6+2}{2} = \mathbf{4}$	$\dfrac{6-2}{2} = \mathbf{2}$

Bảng 16: *Bảng tính thành phần chẵn và lẻ của $x(n)$*

Bước 4 - Kiểm tra

$$x_e(n) + x_o(n) = [4-2,\ 4-1,\ 4+0,\ 4+1,\ 4+2] = [2,\ 3,\ 4,\ 5,\ 6] = x(n) \quad \checkmark \tag{1}$$

$$x_e(-n) = [4,\ 4,\ 4,\ 4,\ 4] = x_e(n) \quad \text{(chẵn)} \quad \checkmark \tag{2}$$

$$x_o(-n) = [2,\ 1,\ 0,\ -1,\ -2] = -x_o(n) \quad \text{(lẻ)} \quad \checkmark \tag{3}$$

Nhận xét:

- $x_e(n) = 4$ là tín hiệu **DC hằng số** - thành phần chẵn thuần túy
- $x_o(n) = n$ tăng tuyến tính - phản đối xứng hoàn hảo quanh $n = 0$.

5.4 Problem 2.6: Time Invariance of Discrete-Time Systems

Yêu cầu đề bài

- **Xét hệ thống rời rạc:**

$$y(n) = \mathcal{T}[x(n)] = x(n^2)$$

- **Thực hiện các yêu cầu sau:**

  (a) **Xác định xem hệ thống có bất biến theo thời gian (time invariant) hay không.**

  (b) **Để làm rõ kết quả ở phần (a), giả sử tín hiệu đầu vào:**

$$x(n) = \begin{cases} 1, & 0 \le n \le 3 \\ 0, & \textbf{otherwise} \end{cases}$$

  Thực hiện các bước sau:

  (1) **Vẽ tín hiệu** $x(n)$.

  (2) **Xác định và vẽ tín hiệu đầu ra** $y(n) = \mathcal{T}[x(n)]$.

  (3) **Vẽ tín hiệu dịch** $y_1(n) = y(n-2)$.

  (4) **Xác định và vẽ tín hiệu đầu vào dịch** $x_2(n) = x(n-2)$.

  (5) **Xác định và vẽ tín hiệu đầu ra** $y_2(n) = \mathcal{T}[x_2(n)]$.

  (6) **So sánh hai tín hiệu** $y_2(n)$ **và** $y(n-2)$. **Đưa ra kết luận.**

  (c) **Lặp lại phần (b) cho hệ thống:**

$$y(n) = x(n) - x(n-1)$$

  Hãy cho biết hệ thống này có bất biến theo thời gian hay không

  (d) **Lặp lại phần (b) và (c) cho hệ thống:**

$$y(n) = \mathcal{T}[x(n)] = n\,x(n)$$

(a) - Kiểm tra tính bất biến thời gian

$$y(n) = \mathcal{T}[x(n)] = x(n^2)$$

Hệ thống $\mathcal{T}$ được gọi là **bất biến thời gian (Time Invariant)** khi và chỉ khi với mọi tín hiệu vào $x(n)$ và mọi số nguyên k, nếu:

$$x(n) \xrightarrow{\mathcal{T}} y(n)$$

thì:

$$x(n-k) \xrightarrow{\mathcal{T}} y(n-k)$$

Nói cách khác, một sự **trễ ở đầu vào** phải tạo ra một sự **trễ tương ứng ở đầu ra**, không làm thay đổi hình dạng tín hiệu. Điều kiện cần kiểm tra:

$$\mathcal{T}[x(n-k)] = y(n-k) \qquad \forall\, n,\, k \in \mathbb{Z}$$

Phân tích

Đại lượng 1: Trễ tín hiệu đầu vào k đơn vị, rồi đưa qua hệ thống $\mathcal{T}$:

$$y(n,\,k) \;=\; \mathcal{T}[x(n-k)] \;=\; x\big((n-k)^2\big) \tag{1}$$

Đại lượng 2: Đưa tín hiệu qua hệ thống $\mathcal{T}$ trước, rồi trễ đầu ra k đơn vị:

$$y(n-k) \;=\; x(n^2)\Big|_{n\,\to\,n-k} \;=\; x\big(n^2-k\big) \tag{2}$$

So sánh

Từ (1) và (2), khai triển $(n-k)^2$:

$$y(n,\,k) = x(n^2 - 2nk + k^2) \tag{1}$$

$$y(n-k) = x(n^2 - k) \tag{2}$$

Giả sử (1) = (2), ta cần:

$$\begin{aligned} n^2 - 2nk + k^2 \;&=\; n^2 - k \\ \Leftrightarrow \quad k^2 - 2nk + k \;&=\; 0 \\ \Leftrightarrow \quad k\,(k - 2n + 1) \;&=\; 0 \end{aligned}$$

Phương trình này chỉ nghiệm đúng khi $k = 0$ hoặc $k = 2n - 1$, tức là **không thỏa mãn với mọi** $n \in \mathbb{Z}$ và $k \in \mathbb{Z}$. Do đó:

$$\boxed{y(n,\,k) \;\neq\; y(n-k) \qquad \forall\, n,\,k \in \mathbb{Z}}$$

Kết luận

$$\mathcal{T}[x(n-k)] \;\neq\; y(n-k) \qquad \Longrightarrow \qquad \textbf{Hệ thống KHÔNG bất biến theo thời gian}$$

Nhận xét: Nguyên nhân là do đối số n^2 là hàm **phi tuyến** của n, khiến phép dịch thời gian ở đầu vào không tương đương với phép dịch thời gian ở đầu ra. Tổng quát, hệ thống dạng $y(n) = x\big(f(n)\big)$ với $f(n) \neq n + c$ (c là hằng số) luôn là hệ thống **biến đổi theo thời gian**. ■

(b) - Minh họa bằng tín hiệu cụ thể

Để làm rõ kết quả ở phần (a), giả sử tín hiệu đầu vào:

$$x(n) = \begin{cases} 1, & 0 \le n \le 3 \\ 0, & \text{otherwise} \end{cases}$$

được đưa vào hệ thống. Thực hiện các bước sau:

(1) Vẽ tín hiệu $x(n)$.

Tín hiệu $x(n)$ nhận giá trị bằng 1 tại $n \in \{0, 1, 2, 3\}$ và bằng 0 ở tất cả các vị trí còn lại:

n	-2	-1	0	1	2	3	4	5
$x(n)$	0	0	**1**	**1**	**1**	**1**	0	0

Bảng 17: *Giá trị tín hiệu đầu vào $x(n)$*

(2) Xác định và vẽ tín hiệu đầu ra $y(n) = \mathcal{T}[x(n)]$.

Tính $y(n) = x(n^2)$: cần $0 \leq n^2 \leq 3$, tức $n \in \{-1, 0, 1\}$:

n	-3	-2	-1	0	1	2	3
n^2	9	4	1	0	1	4	9
$y(n) = x(n^2)$	0	0	**1**	**1**	**1**	0	0

Bảng 18: *Giá trị tín hiệu đầu ra $y(n) = x(n^2)$*

$$y(n) = \begin{cases} 1, & n \in \{-1, 0, 1\} \\ 0, & \text{otherwise} \end{cases}$$

(3) Vẽ tín hiệu dịch $y_1(n) = y(n-2)$.

Dịch $y(n)$ sang phải 2 đơn vị:

n	-1	0	1	2	3	4	5
$y_1(n) = y(n-2)$	0	0	**1**	**1**	**1**	0	0

Bảng 19: *Giá trị tín hiệu dịch đầu ra $y_1(n) = y(n-2)$*

$$y_1(n) = y(n-2) = \begin{cases} 1, & n \in \{1, 2, 3\} \\ 0, & \text{otherwise} \end{cases}$$

(4) Xác định và vẽ tín hiệu đầu vào dịch $x_2(n) = x(n-2)$.

Dịch $x(n)$ sang phải 2 đơn vị - tín hiệu bằng 1 khi $2 \leq n \leq 5$:

n	0	1	2	3	4	5	6
$x_2(n) = x(n-2)$	0	0	**1**	**1**	**1**	**1**	0

Bảng 20: *Giá trị tín hiệu đầu vào dịch $x_2(n) = x(n-2)$*

$$x_2(n) = x(n-2) = \begin{cases} 1, & 2 \leq n \leq 5 \\ 0, & \text{otherwise} \end{cases}$$

(5) Xác định và vẽ tín hiệu đầu ra $y_2(n) = \mathcal{T}[x_2(n)]$.

$$y_2(n) = \mathcal{T}[x_2(n)] = x_2(n^2) = x(n^2 - 2)$$

Cần $0 \le n^2 - 2 \le 3$, tức $2 \le n^2 \le 5$:

n	-3	-2	-1	0	1	2	3
n^2	9	4	1	0	1	4	9
$n^2 - 2$	7	2	-1	-2	-1	2	7
$x(n^2 - 2)$	0	**1**	0	0	0	**1**	0

Bảng 21: *Giá trị tín hiệu đầu ra $y_2(n) = \mathcal{T}[x_2(n)]$*

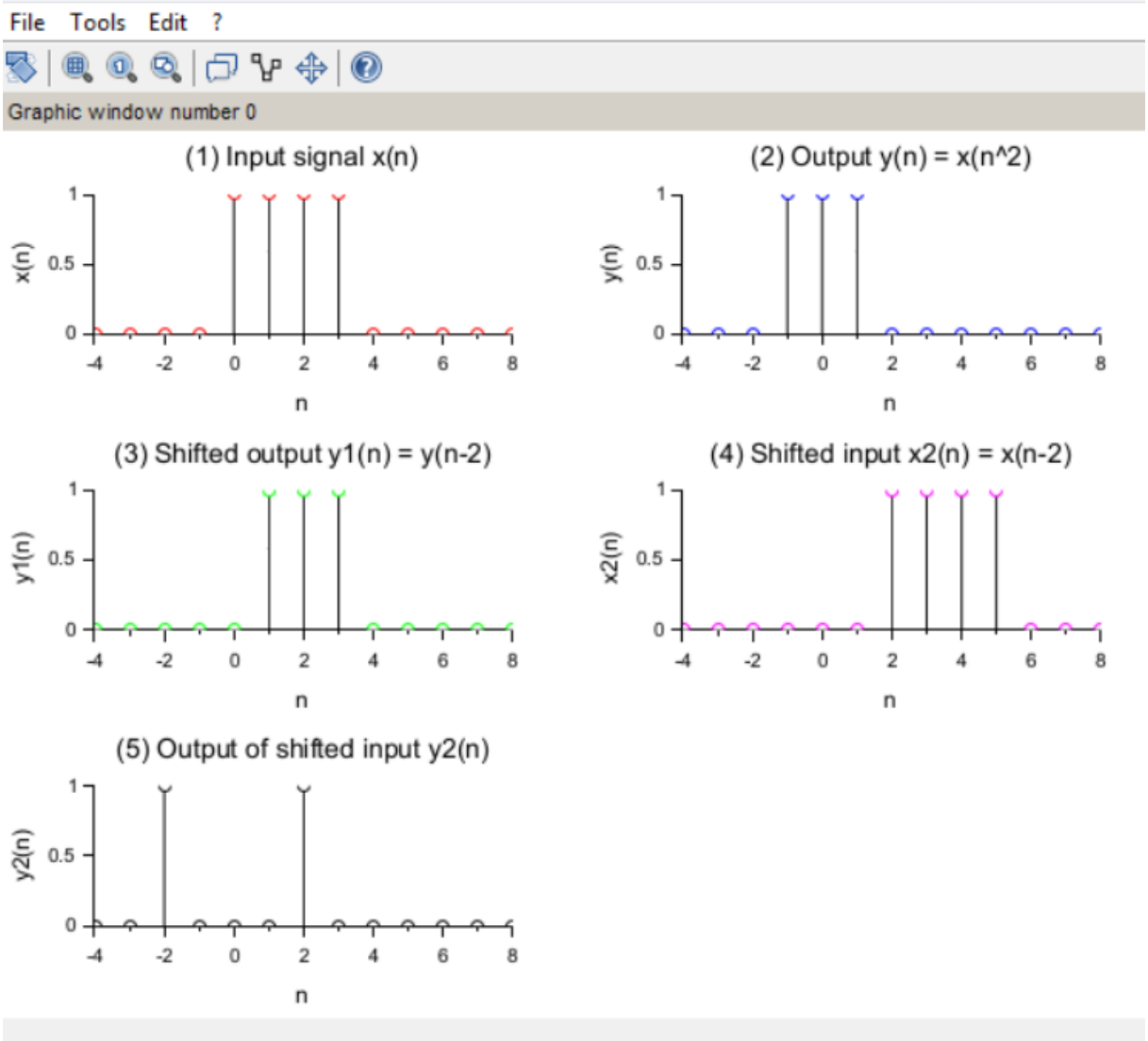
$$\boxed{y_2(n) = \begin{cases} 1, & n \in \{-2, 2\} \\ 0, & \text{otherwise} \end{cases}}$$

(6) So sánh hai tín hiệu $y_2(n)$ và $y(n-2)$. Đưa ra kết luận.

n	-2	-1	0	1	2	3	4
$y_1(n) = y(n-2)$	0	0	0	**1**	**1**	**1**	0
$y_2(n) = \mathcal{T}[x_2(n)]$	**1**	0	0	0	**1**	0	0
Bằng nhau?	×	✓	✓	×	✓	×	✓

Bảng 22: *So sánh $y_1(n) = y(n-2)$ và $y_2(n) = \mathcal{T}[x_2(n)]$*

$$y_2(n) \neq y_1(n) = y(n-2) \quad \Longrightarrow \quad \boxed{\textbf{Hệ thống KHÔNG bất biến theo thời gian}}$$



Hình 19: *Các tín hiệu $x(n)$, $y(n)$, $y(n-2)$, $x_2(n)$ và $y_2(n)$*

Kết luận: Kết quả minh họa hoàn toàn phù hợp với phân tích lý thuyết ở phần **(a)**. Hai tín hiệu $y_2(n)$ và $y(n-2)$ có hình dạng và vị trí hoàn toàn khác nhau, khẳng định rằng hệ thống $y(n) = x(n^2)$ là **biến đổi theo thời gian (Time Variant)**.

(c) - Kiểm tra tính bất biến thời gian của $y(n) = x(n) - x(n-1)$

Hệ thống đã cho:

$$y(n) = x(n) - x(n-1)$$

Phân tích

Đại lượng 1: Trễ tín hiệu đầu vào k đơn vị, rồi đưa qua hệ thống $\mathcal{T}$:

$$y(n,\, k) \;=\; \mathcal{T}[x(n-k)] \;=\; x(n-k) - x(n-k-1) \tag{1}$$

Đại lượng 2: Đưa tín hiệu qua hệ thống $\mathcal{T}$ trước, rồi trễ đầu ra k đơn vị:

$$y(n-k) \;=\; x(n-k) - x\big((n-k)-1\big) \;=\; x(n-k) - x(n-k-1) \tag{2}$$

So sánh

Từ (1) và (2):

$$y(n,\, k) \;=\; x(n-k) - x(n-k-1) \tag{1}$$

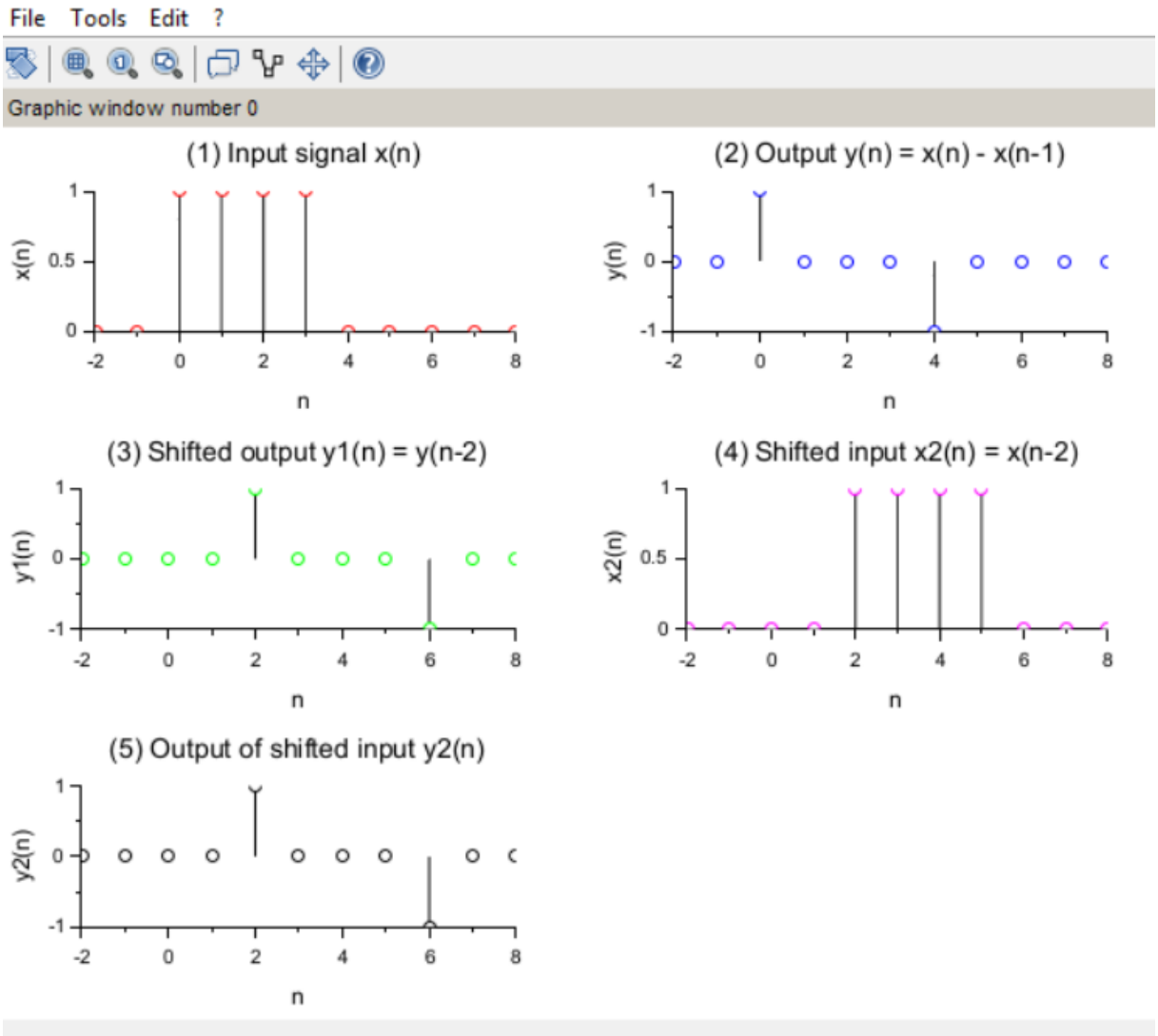
$$y(n-k) \;=\; x(n-k) - x(n-k-1) \tag{2}$$

Hai đại lượng hoàn toàn bằng nhau với mọi n và k:

$$\boxed{y(n,\, k) \;=\; y(n-k) \qquad \forall\, n,\, k \in \mathbb{Z}}$$

Kết luận

$$\mathcal{T}[x(n-k)] \;=\; y(n-k) \qquad \Longrightarrow \qquad \textbf{Hệ thống BẤT BIẾN theo thời gian}$$



Hình 20: *Các tín hiệu $x(n)$, $y(n)$, $y(n-2)$, $x_2(n)$ và $y_2(n)$ với $\mathcal{T}[x(n)] = x(n) - x(n-1)$ và $k = 2$*

Nhận xét: Hệ thống $y(n) = x(n) - x(n-1)$ là phép **sai phân bậc nhất** (first-order difference). Đây là hệ thống tuyến tính bất biến thời gian **(LTI)** vì cấu trúc của nó không phụ thuộc vào thời gian n — chỉ phụ thuộc vào **khoảng cách tương đối** giữa các mẫu tín hiệu. ■

(d) - Kiểm tra tính bất biến thời gian của $y(n) = \mathcal{T}[x(n)] = n\,x(n)$

Hệ thống đã cho:

$$y(n) = \mathcal{T}[x(n)] = n \cdot x(n)$$

Phân tích

Đại lượng 1: Trễ tín hiệu đầu vào k đơn vị, rồi đưa qua hệ thống $\mathcal{T}$:

$$y(n,\,k) \;=\; \mathcal{T}[x(n-k)] \;=\; n \cdot x(n-k) \tag{1}$$

Đại lượng 2: Đưa tín hiệu qua hệ thống $\mathcal{T}$ trước, rồi trễ đầu ra k đơn vị:

$$y(n-k) \;=\; (n-k) \cdot x(n-k) \tag{2}$$

So sánh

Từ (1) và (2):

$$y(n,\,k) \;=\; n \cdot x(n-k) \tag{1}$$

$$y(n-k) \;=\; (n-k) \cdot x(n-k) \tag{2}$$

Giả sử (1) = (2), ta cần:

$$n \cdot x(n-k) \;=\; (n-k) \cdot x(n-k)$$

$$\Leftrightarrow \quad n \;=\; n-k$$

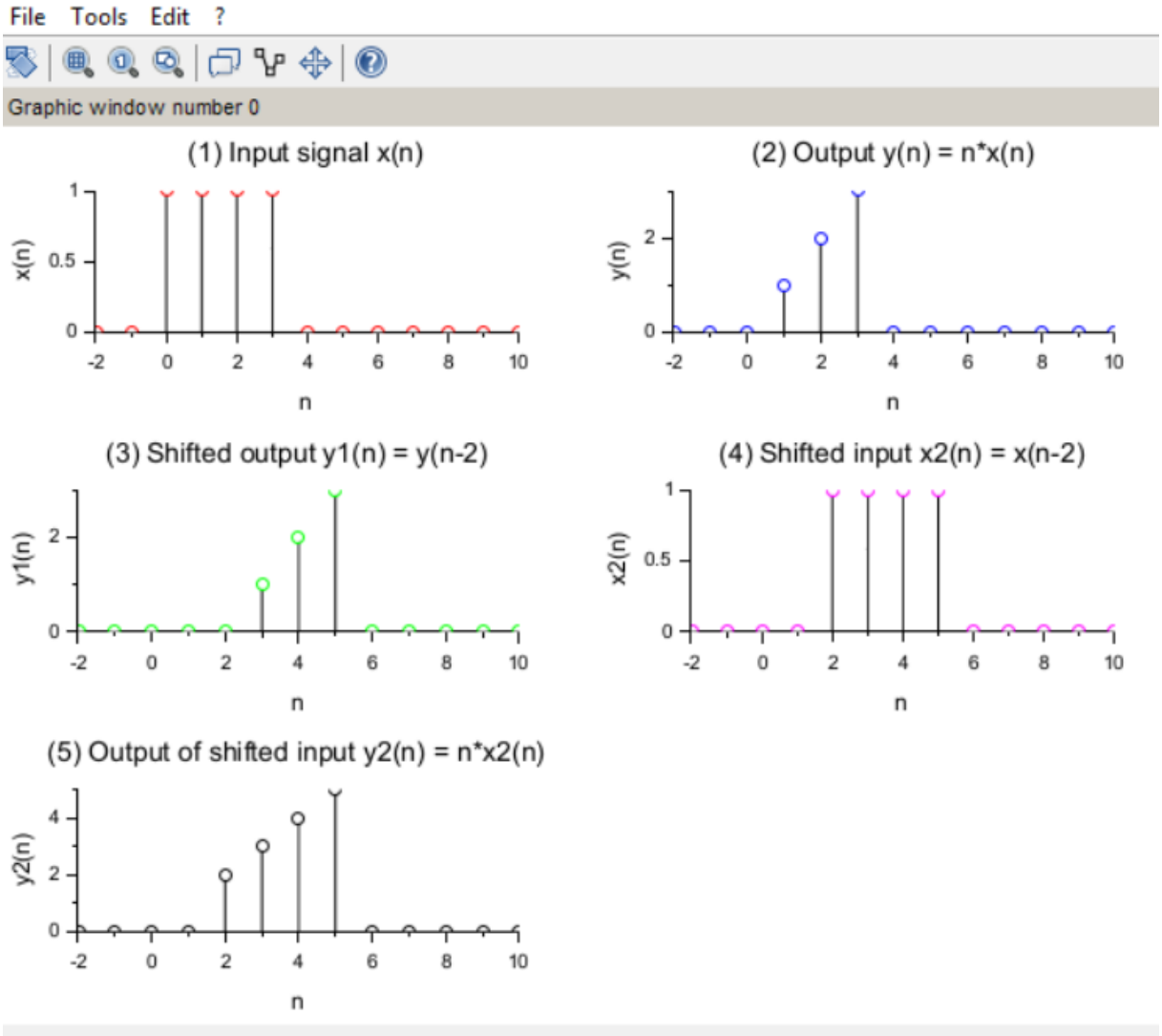
$$\Leftrightarrow \quad k \;=\; 0$$

Đẳng thức này **chỉ đúng** khi $k = 0$, tức là **không thỏa mãn với mọi** $k \in \mathbb{Z}$. Do đó:

$$\boxed{y(n,\,k) \;\neq\; y(n-k) \qquad \forall\, k \neq 0}$$

Kết luận

$$\mathcal{T}[x(n-k)] \;\neq\; y(n-k) \qquad \Longrightarrow \qquad \textbf{Hệ thống KHÔNG bất biến theo thời gian}$$



Hình 21: *Các tín hiệu $x(n)$, $y(n)$, $y(n-2)$, $x_2(n)$ và $y_2(n)$ với $\mathcal{T}[x(n)] = n \cdot x(n)$ và $k = 2$*

Nhận xét: Hệ số nhân n thay đổi theo thời gian, khiến hệ thống phụ thuộc trực tiếp vào vị trí thời gian n. Đây là dấu hiệu điển hình của hệ thống **biến đổi theo thời gian (Time Variant)** — hệ số của tín hiệu vào không phải hằng số mà là hàm của n.

Tài liệu

[1] Scilab Enterprises. (2010). *Scilab for beginners*. Orsay, France: Scilab Enterprises. Available at: https://www.scilab.org/sites/default/files/Scilab_beginners.pdf

[2] Scilab Enterprises. (2024). *Scilab — Open source software for numerical computation*. Available at: https://www.scilab.org

[3] Proakis, J. G., & Manolakis, D. G. (2007). *Digital signal processing: Principles, algorithms, and applications* (4th ed.). Upper Saddle River, NJ: Pearson Prentice Hall.

[4] Link Github. (2025). *Signal Processing Exercises using Scilab*. GitHub repository. Available at: https://github.com/anhthu057/XU_LY_TIN_HIEU_SO/tree/lab3